重庆大学本科学生毕业设计（论文）

基于深度学习的活动感知算法研究与实现

学　　生：项怡雯

学　　号：20183749

指导教师：古富强

专　　业：计算机科学与技术

重庆大学计算机学院

2022 年 6 月

Graduation Design(Thesis) of Chongqing University

Research and Implementation of Activity Recognition Algorithm Based on Deep Learning

Undergraduate: Xiang Yiwen

Supervisor: Prof. Gu Fuqiang

Major: Computer Science and Technology

School of Computer Science

Chongqing University

June 2022

摘 要

目前，对于深度学习的研究成为了热点，学者们开始致力于将深度学习应用于不同的研究领域,在活动识别领域深度学习的相关应用也开始出现。这篇文章主要总结了前人在活动识别领域研究的缺陷，并在一般 LSTM 网络的基础上，提出了 DeepHAR 网络。新网络的优点主要是通过增加残差连接（在深度维度上）和双向单元（在时间维度上）保证了信息传输的有效性，从而可以将底层信息自下而上进行传输，避免梯度消失的问题，最终提高了模型的效果。文章详细对比了不同网络在不同数据集的表现，最终的网络在自建智能手表数据集和公开 UCI HAR 数据集中取得了 86.69%和 93.34%的准确率。文中还介绍了一个能够在 Ticwatch Pro 3 手表上应用的智能手表传感器数据收集系统。通过该系统，可以在不同频率、不同活动标签下对手表内传感器陀螺仪和加速度计的数据变化进行采集、初步处理并存储，有利于未来基于智能手表数据的活动识别研究。

关键词：活动识别，长短期记忆人工神经网络，残差网络，传感器

ABSTRACT

Deep learning has become relatively popular nowadays. Scholars begin to devote themselves to applying deep learning in various aspects, and the application of deep learning in the field of human activity recognition(HAR) also appears. This paper summarizes the shortcomings of previous research in the field of activity recognition, and proposes a new network, DeepHAR network, on the basis of simple LSTM. The new network can ensure the effectiveness of information transmission by adding residual connections (in depth dimension) and bidirectional units (in time dimension). In this case, the bottom information can be transmitted from bottom to bottom, avoiding the gradient disappearance problem. Finally, the model's performance can be improved. Also, the performance of different networks in different data sets is compared in detail in the paper. The final network achieves 86.69% and 93.34% accuracy in the self-built smartwatch data set and public UCI HAR data set. The paper also introduces a smartwatch sensor data collection system that can be used on Ticwatch Pro 3. Through this system, data from the gyroscope and accelerometer in the watch can be collected, preliminarily processed, and stored at different frequencies and active labels, which is conducive to future research on activity recognition based on smartwatches.

Key words： activity recognition, LSTM, residual network, sensors

目 录

中文摘要 I

ABSTRACT II

1 绪论 1

1.1 课题研究背景及意义 1
1.2 活动识别概述 2
1.3 国内外研究现状及分析 2
1.3.1 活动识别的研究方向 2
1.3.2 活动识别的数据获取 3
1.3.3 活动识别的现有研究方法 5
1.3.4 深度学习方法中神经网络模型的选择 7
1.3.5 研究重难点 8
1.4 本文主要研究内容 9
1.5 论文章节安排 9

2 手表传感器数据获取系统 11

2.1 系统框架设计 11
2.1.1 系统基本要素与设备要求 11
2.1.2 系统功能要求和流程 11
2.2.系统的实现 12
2.2.1 原始数据获取 12
2.2.2 数据处理：卡尔曼滤波器 13
2.2.3 基于 FTP 协议的文件存储 16
2.3 系统展示 16

3 活动识别算法研究 20

3.1 技术路线 20
3.2 数据集 21
3.2.1 智能手机数据集 21
3.2.2 智能手表数据集 22
3.3 数据处理 23
3.4 基于深度学习的活动识别算法 24
3.4.1 基础的 LSTM 结构 24
3.4.2 双向 LSTM 结构 26
3.4.3 残差网络 26
3.4.4 深度残差的双向 LSTM 网络 28

3.4.5 超参数设定方法 31
4 实验与结果 32
4.1 超参数选择 32
4.2 算法在智能手表数据集上的效果 32
4.2.1 数据集分析 32
4.2.2 基础的 LSTM 算法 34
4.2.3 优化算法 37
4.3 算法在智能手机数据集上的效果 40
4.3.1 数据集分析 40
4.3.2 基础的 LSTM 算法 41
4.3.3 优化算法 42
4.4 算法效果比较 46
4.4.1 两个数据集的结果比较 46
4.4.2 其他结果比较 46
5 结论与展望 48
致谢 50
参考文献 51

1 绪论

1.1 课题研究背景及意义

人们在目前的生活中已经离不开多种多样的智能设备，包括智能手机、智能手表等。多种多样的智能设备和内置其中的传感器为人类的生活提供了多种便利，而利用传感器数据分析人们的行为轨迹和行为特征成为了一个重要的研究课题。

运动和健身离不开对人们活动的研究。调查显示，有 80%的大学生的手机中安装有“悦跑圈”等软件，这些软件通过收集内置在手机中的传感器信息，能够分析人们在运动时的活动状态，记录运动信息，进行周期化反馈，进而便于用户管理自己的运动和健康状态。这些健康运动软件对用户活动的的分析即人类活动识别分析的一种分支。人类活动识别 (HAR) 是一个重要的研究课题，其在人们的日常生活中发挥着重要作用，活动感知的研究对于定位导航、健康监控、老年人关爱等应用都具有重要意义。当前主流的智能设备（例如智能手机、手表）等已内置了加速度计、陀螺仪、磁力计等多个与人类活动有关的传感器，而基于这些传感器数据进行数据处理和分析后，我们可以进一步采用机器学习、深度学习等算法找到不同人类活动的性质和特点，从而对人们生活中的不同活动进行判别和分析，最后进一步开发上层应用。

机器学习是人工智能的核心技术之一，而近几年发展起来的深度学习已成为当前主流的机器学习方法。基于深度学习的活动感知已成为当前的研究热点之一，但其还存在如下挑战：

1）深度学习网络往往需要大量的训练数据以避免模型过度拟合；

2）深度学习网络的计算量很大，往往需要在高性能的服务器上执行；

3）人们的行为具备高度随机性，因此活动识别的精度受限。

本课题研究旨在根据上述挑战，提出新型的轻量、精确的活动识别方法，通过自学习的深度学习技术减少训练模型所需的数据量，并研究模型优化技术提高计算效率。

通过本文的研究，可以将通过智能手表收集的数据使用深度学习技术进行模型训练和精确活动识别，在减少模型训练量的同时提高识别的精度。同时，最终实现的活动识别算法可以用于判断人体的运动状态，应用到运动、健康等多个领域，在未来更可以嵌入到手机、手表等不同设备中，具有较高的研究和实用价值。

1.2 活动识别概述

人类活动识别 (HAR) 在我们的生产和日常生活中都起到了非常大的作用，它能够分析从原始传感器输入中获得的数据，从而学习有关人类活动的重要知识[1]，并涉及基于传感器数据识别人的特定运动或动作的广泛研究领域，是多个学者的重点研究方向。人类活动实际上是多种多样的，包括运动（例如，步行、跑步、站立和静止）、运动（例如，骑自行车和踢足球）、与健康相关的活动（例如，跌倒、康复和遵循常规）、日常活动（例如，购物、使用电脑、睡觉、上班和参加会议）等[2]。事实上，人类活动可以被科学地分为三种不同的类型[3]。

第一种活动类型：手势

指一种简单的手部动作或人体其他部位动作用于传达一种想法。其包括了面部表情、挥手和摇头。手势通常在短时间内执行，它是最简单的活动。

第二种活动类型：动作

动作是由人类执行并涉及多个手势的简单活动。动作的例子有敲门、游泳和跑步等。

第三种活动类型：交互

交互是由两个对象进行的活动。人是其中一个对象之一，而另一个对象可以是物体，也可以是人。摔跤、拥抱和握手都是人与人交互的例子，而人与物体交互的例子是人与手机或笔记本电脑之间的交互。

人类活动识别就是对于这三种人类活动的识别。通过对于三种活动中细分的的小活动分析特点，对每个不同的人类动作作出判断，这是活动识别的具体流程。从应用来说，应用在我们生活中的活动识别就包括居家行为分析[4]、视频监控分析[5]、步态分析[6]和手势识别[7]等。这些活动的识别，除了能够为人们的生活提供便利之外，还能够在室内定位、能量消耗、健身追踪和老年护理等多个方面起到不可缺少的作用[2]。

1.3 国内外研究现状及分析

活动识别在国内外目前都是不少学者研究的核心之一，学者们已经从多个不同的角度对活动识别进行了研究，本章节将从不同角度对于活动识别的目前研究趋势和未来的研究方向进行分析。

1.3.1 活动识别的研究方向

综合了所有可以收集到的文献，我们首先可以根据不同来源的活动识别数据对活动识别的类型进行区分。事实上我们可以把活动识别分成以下两类：基于环境感知方法和基于可穿戴感知方法的活动识别。

基于环境感知方法的活动识别主要通过分析来自基础设施（例如，摄像机网络或无线接入点）的数据来检测人类活动[2]。例如，现在闭路电视系统可以利用计算机视觉技术来检测人类活动，进行特征提取、运动检测和轨迹分析[8]。该方法也可以利用无线收发器覆盖区域内的无线信号，根据信号的衰减、衰落和传播特性识别人类活动[9]。同时，我们也可以通过在环境中安装摄像机，对来自摄像机的包含人体运动的视频或图像进行处理来进行人的行为分析[10]，这一种活动识别也被称为基于视觉的活动识别，其已经被研究了数十年[8][11]，涉及标记具有不同活动的图像序列或视频。总的来说，基于环境的活动识别分析方法非常丰富，应用也非常广泛。但是，由于基于环境的活动识别的覆盖区域有限，并且难以对人类的连续活动进行识别，因此其在应用时往往会具有一定的局限性[2]。

基于可穿戴感知方法的活动识别通常是利用现代智能设备[12]中内置的传感器来获得数据的。现代智能设备集成了许多可用于活动识别的传感器，包括加速度计、陀螺仪、磁力计、气压计、麦克风和光传感器等传感器。在这些内置于智能设备中的传感器中，加速度计是使用最广泛的传感器之一[13]，因为它可以识别各种活动。而其他运动相关传感器[14]陀螺仪、磁力计、气压计等，常联合使用，来补充加速度计的数据。目前，利用可穿戴感知方法的活动识别正变得越来越流行并被广泛利用。主要原因之一就是基于穿戴的传感器没有覆盖范围限制，因此可以在任何地方工作；同时，由于智能手机、智能手表和智能眼镜等富含传感器的智能设备的出现，可穿戴传感器的使用范围变得更加广泛了；除此以外，传感器技术及普适计算技术也正在蓬勃发展。作为未来活动识别的发展方向之一，基于可穿戴感知方法的活动识别逐渐成为了学者们主流的研究方向。在本课题中，我们也会更关注基于传感器的活动识别，研究如何利用更加先进的方法来提升活动识别的精度。

1.3.2 活动识别的数据获取

活动识别研究的数据主要来自于不同的传感器，根据选择研究的活动识别方向不同，应该选择不同类型的传感器获得数据。目前主流的人类活动识别传感器主要有三类：可穿戴传感器、环境传感器和其他传感器。

（1）环境传感器

环境传感器需要安装在固定位置才能识别活动，通常包含一个服务器（例如，WiFi 接入点）和一个客户端（例如，WiFi 接收器）[15]。其大都是在智能家居中被利用[7]，来进行对环境感知的活动识别研究。常见的环境传感器有全球导航卫星系统、蜂窝、无线网络、射频识别等[15]。

全球卫星系统：通过 GNSS（全球导航卫星系统）的可用卫星来识别物体的

位置、移动速度，最后可以成功识别人类的运动、交通方式和日常活动。手机 APP 中的导航 APP 就是依赖卫星观测到的人们的位置进行定位、分析，从而判断人们的运动速度和运动状态。该传感器不仅仅在我们的生活中得到有效的应用，在灾难救援、极端环境探索等领域都有大量的应用。

蜂窝网络：蜂窝网络是目前移动通信中常用的通信网络。研究者们可以通过测量蜂窝接收器和之间的接收信号强度发射器来识别不同的活动。其识别准确度甚至能够达到加速度计所能达到的精度。

无线网络：无线网络是一种局域无线通信技术，它可以被用于测量活动识别中的往返时间等属性。由于无线网络目前在城市和室内环境中都已经被广泛应用，因此对其的研究也较为丰富。曾经就有学者们在手机上使用 WiFi 来识别人们的状况、活动和手势[16]。

射频识别：射频识别通过阅读器与标签进行数据通信，这种通信是非接触式的。其根据通信结果对目标进行识别并被应用在非常广泛的领域，比如动物晶片、汽车防盗器、门禁管制等都。学者们也提出过一种使用深度卷积从无源射频识别数据中识别活动的系统神经网络[17]。

环境传感器虽然应用很广泛，但是其识别有区域范围限制，只能在特定环境中识别人的活动。尽管卫星可能不限制识别的区域，但是其成本很高，且大部分数据属于个人隐私，很难收集得到，因此对环境传感器数据的获取和研究都有一定局限性和难度。

（2）可穿戴传感器

在本课题中，我们主要会研究基于可穿戴传感器的活动识别。我们将通过分析传感器的数据来判断人们的活动，研究重点将会在可穿戴传感器产生的数据上。可穿戴传感器包括加速度计、陀螺仪、磁力计、气压计、麦克风和光传感器等，其中加速度计是经常被应用于研究的一个重要传感器，它可以记录人运动时的速度。我们将加速度计配合其他传感器，如可以测量方向和角速度的陀螺仪、测量磁场的磁力计等内置在手机、手表中，从而记录人们的活动数据，然后分析得到人们的运动状态。除了一些内置在设备中的传感器之外，相机也是可穿戴传感器的一种，但是与其他传感器分析数据不同，对相机数据的分析，是对相机获得的视频的分析，相比较数据分析更类似于计算机的图像处理：通过比对固定时间间隔中人的姿态，用图像识别处理技术判断人的运动状态。本文主要研究对加速度计等内置可穿戴传感器的数据分析，因此相机等类似图像传感器数据研究在本次论文的讨论范围之外。

针对内置在移动设备中的传感器，目前应用比较广泛的分为手机和手表两种，实际上手机中的传感器数据获取已经较为完备了，安卓手机中有 AndroSensor 等

APP，苹果手机中有 Sensors 等 APP。这些 APP 都可以非常便利地获取人们手机中传感器的数据。但实际上，智能手表的传感器数据获取还相对空缺，大多数研究都是直接对智能手环、手表获取的数据进行研究，智能手表相关的活动识别研究还较为贫乏。目前已有的对智能手表的传感器分析使用的方法也都是传统的模式识别的方法，如 Serkan Balli 等人就使用主成分分析和随机森林算法混合的方法进行了智能手表传感器数据的人类活动识别[18]，该方法较为传统，作者也承认无法实时进行应用，因此我们需要用深度学习的方法对传统的方法进行改良来对智能手表的数据进行分析。同时，由于智能手表的类型众多，我们也无法直接使用他人获得智能手表中传感器数据的方法，需要开创我们自己的道路。

（3）其他传感器

其他传感器中的一些类型实际与相机类似，但是不同的是，这部分摄像机会输出异步事件，而不是传统同步的内容，是一种较新的设备，因此研究的资料也相对较为缺少，设备也很难获得。

1.3.3 活动识别的现有研究方法

（1）传统的模式识别方法

很多学者通过传统的模式识别方法来对活动识别的数据进行分析。例如，他们会利用传统且典型的方法如决策树、支持向量机、朴素贝叶斯和隐马尔可夫模型等算法解决这一类问题，提高活动识别的精度[19]，具体的分析过程如图 1.1 所示。虽然这些传统分类器可用于识别不同的活动，但它们的性能主要取决于数据手工特征的适用性。这些功能的设计需要专业知识，是一项艰巨而费力的任务。例如，要在 HAR 中获得令人满意的精度，就必须考虑提取不同类型的特征，例如统计特征（如均值、方差等）、频域特征（如傅里叶变换）和时域特征（如自相关系数）。根据传统方法，原始信号输入是从多种类型的传感器（智能手机、手表、Wi-Fi、蓝牙、声音等）获得的。然后人类手动提取一些模型的特征[20]，例如传统机器学习方法中的均值、方差[21]等。在完成繁重的特征提取任务之后，这些特征被用作训练 PR 模型的输入，在后面的模型中起到了不可忽视的作用。这里的特征选择方法变得非常重要且复杂[2]。因此，在一些只需要少量标记数据的环境中，传统的分析方法确实能够获得令人满意的效果。但是也可以明显发现这种方法的弊端，通过手工特征提取重要特征，极易由于人的感觉引起误差，而由有误差的特征总结计算分析得到的结果往往与实际的结果存在大量的偏移。因此，在日常活动识别任务中，这种严重依赖于手工的特征提取且只能学习浅层特征[22]的传统方法，会导致活动识别任务的性能和精度下降，可以总结得到这种传统的分析方法在人类日常活动识别和大量数据的环境中并不适用。

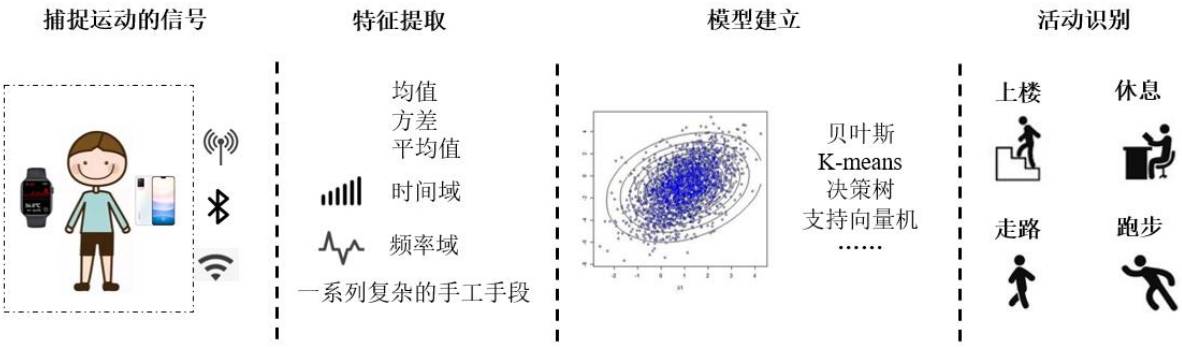


图 1.1：基于模式识别方法的活动识别分析过程

（2）深度学习方法

为了解决传统的活动识别研究方法的局限性，新的研究方法应运而生。近年来，深度学习技术快速发展，并在视觉对象识别、自然语言处理和逻辑推理等领域都取得了广泛应用，更展现出了非常高的精度和强劲的性能[22]。因此，一部分专家学者开始考虑将深度学习应用在活动识别方面。

如图 1.2，对于深度学习来说，特征提取和模型构建过程通常同时进行。这些特征都是可以通过网络自动学习获得的，而不是由人自行设计，从而提高了结果的科学性。此外，深度神经网络还可以提取深层特征，从而处理更复杂的活动识别任务。当输入的网络的数据都缺少标注时，深度神经网络也可以通过这些已有的未标记数据进行模型训练[1]。总的来说不同于传统的模式识别方法，深度学习可以大量减轻设计特征的工作量，通过自己的网络提取特征，同时可以通过训练神经网络来学习更多隐藏在数据中不易被发现的的特征。深度学习也可以进行无监督和增量学习，提高算法的准确度。因此，深度学习在现有工作中得到了广泛的探索和较多的应用[23] [24]。

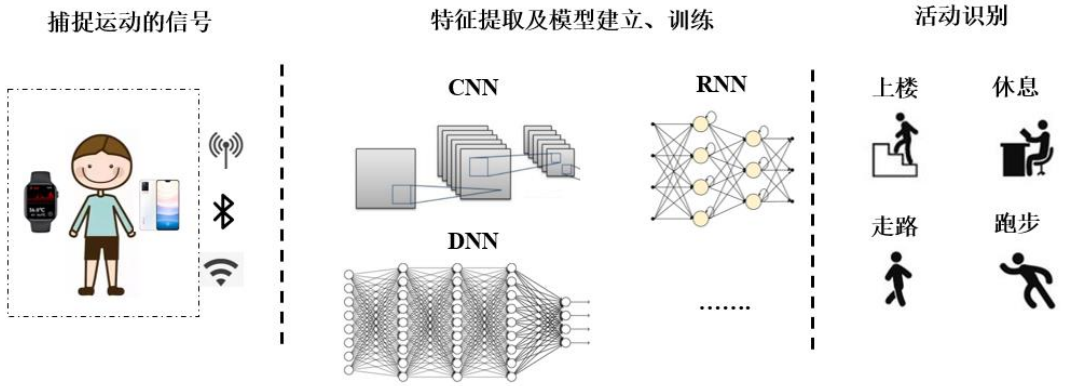


图 1.2：基于深度学习方法的活动识别分析过程

（3）模式识别方法和深度学习方法的比较

在数据预处理方面，模式识别方法需要进行彻底的数据预处理和规范方法，而深度学习方法不需要。在特征提取方面，模式识别方法依赖于手动提取特征且无法处理复杂的活动，但是深度学习方法可以自动从原始数据中学习抽象的特征。在学习过程方面，模式识别方法相比于深度学习耗时较长，且需要消耗大量内存。但是深度学习方法也有缺点，其需要大量的数据集进行输入防止过度拟合，计算的复杂度也非常高，因此有一定的硬件要求[3]。但是，总体而言，将深度学习应用于活动识别克服了传统方法的缺点，能够高度提炼特征，提高模型准确度，因此我们在本课题中放弃了传统的模式识别方法，选择了深度学习的方法来建立活动识别的模型和进行数据训练。

1.3.4 深度学习方法中神经网络模型的选择

深度学习是机器学习方法的一个子集，具有多个层次。主流的深度学习方法包含深度神经网络（DNN）、卷积神经网络（CNN）、递归神经网络（RNN），它们都是在如今学者们进行训练里常用到的网络。

（1）深度神经网络

深度学习网络是具有多个隐藏层的人工神经网络,因此深度学习网络也被称为深度神经网络[15]。其优点在于可以自动分析海量数据，从数据中得到相似的特征进行学习，并且该能力随着网络层数的增多而增强。

在活动识别的时候应用该方法主要会从传感器中提取手工设计的特征，然后将这些特征输入到 DNN 模型中。在这些工作中，DNN 仅作为手工特征提取后的分类模型，因此它们可能无法很好地泛化，而且网络很浅。一旦需要处理多维且复杂的活动识别数据时，较浅的网络往往无法取得良好的效果，而更多的隐藏层更深的网络可以帮助模型训练得更好，具备更强的能力 [25]。但是，在某些情况下应考虑更多细节，以帮助模型更好地微调。

（2）卷积神经网络

卷积神经网络在图像分类、语音识别和文本分析的多个方面已经取得了广泛的应用。其能够从信号中提取特征的优势被很多深度学习相关的活动识别工作所利用，比如 Baoding Zhou 就曾提出一种基于智能手机的基于卷积神经网络的活动识别方法，该方法可用于室内定位且可以自动学习特征，从而避免耗时的特征提取和选择，其模型的精确度高达 98%，足见其有效性[26]。在使用时，其局部依赖性和尺度不变性提高了模型的准确率。其中局部依赖性代表活动识别时任意某个数据和其附近的信号可能是相关联的，而尺度不变性是指不同步速或频率的尺度

不变性[1]。

（3）递归神经网络

递归神经网络的研究主要是利用了神经元之间的时间相关性，其中的重要部分是 LSTM（长短期记忆人工神经网络）。在实际应用中，它通常与递归神经网络结合，LSTM 用于梯度下降来作为记忆单元。在了解递归神经网络后可以发现，由于其学习速度慢、资源消耗较大，很少有工作直接将递归神经网络用于 HAR 任务，但由于 HAR 是时间序列处理任务的性质，LSTM 常被用于活动识别，有学者研究了几个模型参数，提出了一个相对较好的 RNN 模型，可以处理高吞吐量的活动识别数据[27]。

（4）合适的神经网络

在一项研究多个网络在活动识别的论文中，研究者们通过在一些公共活动识别数据集上进行 4000 次实验来研究 DNN、CNN 和 RNN 的性能，他们得出一些结论： RNN 和 LSTM 识别更加适合判别短期活动，而卷积神经网络更擅长推断长期重复活动[6]。这一现象产生的原因是递归神经网络可以利用传感器读数之间的时间顺序关系，而卷积神经网络更能够学习包含在递归模式中的深层特征。同时，他们也指出，在利用 CNN 时，数据驱动的方法比模型驱动的方法更好，因为当输入数据转换为虚拟图像时，可以更好地利用活动信号的内在属性，多重卷积和池化可以更好地提高神经网络的性能。

因此我们在课题研究中，会比较一些较多使用的网络进行数据的训练，如 LSTM、ResNet 等前沿网络。由于在不同的数据集上，采用不同的模型会有非常不同的效果，因此更加具体的网络选择，会在评估不同模型在数据集中的效果后进行确定，选择出最适合我们数据集的网络。

1.3.5 研究重难点

虽然已经有很多学者在活动识别领域进行研究，但是基于深度学习的活动识别的研究仍然存在挑战，需要我们在课题研究中进行进一步的认知和解决。

（1）基于手表数据的活动识别

在目前已有的研究的基础上，本课题将会针对人佩戴手表时的行为进行活动识别。与传统较为单一的活动识别不同。传统的活动识别的数据通常来自于手机，大部分数据库也来自于手机数据。但是在我们的研究过程中，将利用手表进行数据收集，使最后获得的数据更贴近人们的日常生活，更具有参考价值。

（2）多个传感器的利用

混合传感器提供的详细信息对于识别细粒度活动很有用[4]。因此，我们在研究中可以注意结合多个传感器的数据进行训练，而非传统的一个传感器的数据。

（3）轻量级深度模型

深度学习的模型通常需要大量的计算资源，而这对于可穿戴设备来说是不可用的。此外，模型通常是离线训练的，无法实时执行。然而，较不复杂的模型，如浅层神经网络和传统的 PR 方法无法获得良好的性能。因此，我们有必要在保证准确度的同时，选择更轻量的深度学习网络来进行研究。

1.4 本文主要研究内容

本文主要是研究基于深度学习的活动感知算法研究与实现。研究的对象是人类活动中的动作类别。数据主要是来自于不同智能设备中的传感器。由于目前几乎所有有关活动识别的研究都是建立在已有的智能手机数据集上，基于智能手表的研究少之又少。因此，在研究后，我们决定自己建立一个智能手表内的传感器数据收集系统，通过实验收集手表中人在不同活动状态下的数据，建立自己的智能手表传感器数据集。本文将选取来自加速度计和陀螺仪的 3 轴原始信号进行收集，数据以 20 dps 的恒定速率捕获，然后使用滤波器滤除噪声，进行数据预处理。最后我们将结合该自行创立的数据集和已有的智能手机数据集，建立深度学习网络，对数据进行训练，对模型的效果进行验证，考虑模型精度和效率等多方面因素，选取出表现最佳的网络算法，便于未来的应用和验证。

1.5 论文章节安排

本文一共分为五个章节，详细介绍了手表传感器数据收集系统和基于深度学习的活动识别算法。

第一章主要是文章的绪论。在本章中，首先提出活动感知的定义和研究现状。接着，主要通过对比过去学者的算法和研究分析得到了目前研究的局限性和不足。最后，说明了本文将解决的活动识别领域存在的问题，并介绍了研究内容和创新性。

第二章介绍了手表传感器数据获取系统的构造、功能和界面。本项目主要通过自己设计的手表传感器获取应用程序来收集智能手表的数据作为数据集之一。本章将详细介绍系统的构造，系统的设计，系统用到的传感器接口和内在算法，并展示最终开发的应用界面。

第三章为本项目用到的算法 DeepHAR 介绍，算法结合了长短期记忆单元、残差网络、双向单元等内容。本章将详细介绍基础网络的构造、优化网络的结构、网络的各种超参数和网络实现的细节，并且详细叙述最终优化的算法。

第四章介绍了算法在两个数据集上的运行的实验步骤和结果。本章比较了不同网络结构在不同数据集的效果，详细介绍了手机和手表两个不同数据集预处理的过程和步骤，并分析了网络的精度和准确性。本文提到了四种模型架构，在研究其在两个数据集上的表现后，总结得到表现最优秀的模型，最后将结果与国内外项目

的研究成果进行对比，指出了本项目算法相比于同一领域其他算法的优势。

第五章总结和展望了本文的内容，该部分主要对文章的所有内容进行了回顾、总结和阐述。从整体上分析了算法和系统的缺陷和导致算法准确率偏低的原因，最终指明了可以改进的方面，并展望了算法未来的应用和下一步的工作。

2 手表传感器数据获取系统

2.1 系统框架设计

2.1.1 系统基本要素与设备要求

本系统主要由前端的展示程序和后端程序构成，要求能够在 TicWatch Pro3 的 Wear OS 系统上进行正常运行。Wear OS 是谷歌 Android 操作系统的一个版本，专为智能手表和其他可穿戴设备设计，其将技术和移动通知集成到一个智能手表，并提供了多个有关传感器的接口可供开发者使用。同时，手表所采集到的数据将在 Windows 电脑上进行同步，从而实现数据传输，便于后续的数据处理。

2.1.2 系统功能要求和流程

本系统主要功能是实现实时记录智能手表内传感器数据，并将数据传输于电脑，为后续算法的搭建和效果测试作准备。

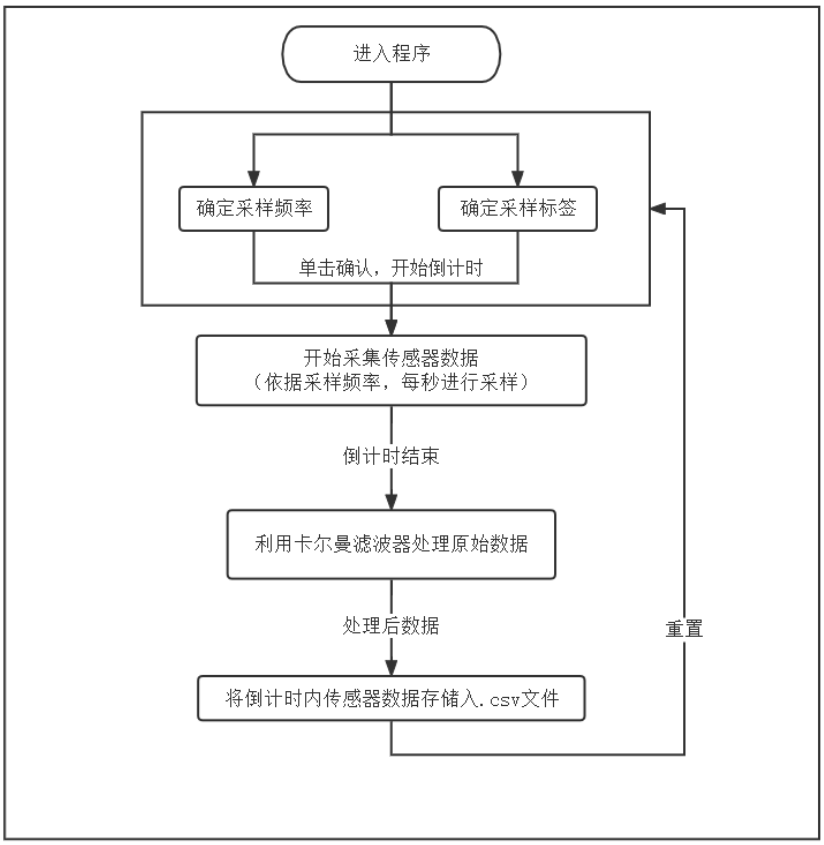


图 2.1：传感器数据采集系统系统设计图

设计的整体思路是：首先通过访问智能手表系统 WearOS 中的传感器接口，获得传感器数据。接着程序将通过倒计时限制记录倒计时时间内传感器的状态，并将传感器数据标上标签经卡尔曼滤波器处理后存储进入文件。在倒计时结束后数据将被存储为一个.csv 文件，存在手表上。具体的系统设计图如图 2.1 所示。

2.2. 系统的实现

2.2.1 原始数据获取

由于 WearOS 系统类似安卓系统，因此在设计系统时，主要采用了 Andriod Studio 进行设计，具体的设计过程与设计安卓 APP 流程类似。虽然 WearOS 是在 2014 年推出的，至今已有八年，但是国内厂家近几年才开始适配并进行生产，因此有较大的未来发展空间与发展潜力。

项目的重点在于找到传感器的接口。大多数 Android 设备都有内置传感器，用来测量运动、屏幕方向和各种环境条件。这些传感器能够提供高度精确的原始数据，非常适合用来监测设备的三维移动或定位，或监测设备周围环境的变化。我们的实验数据主要需要用到加速度计和陀螺仪的数据，因此我们通过监测相应的传感器接口获得相应的实验数据，具体接口如表 2.1 所示。

表 2.1：传感器及其接口

传感器名称	传感器接口	传感器向量构成	变量名	单位
加速度计	Sensor.TYPE_ACCELEROMETER	SensorEvent.values[0]	acc_x	m/s^2
		SensorEvent.values[1]	acc_y	
		SensorEvent.values[2]	acc_z	
陀螺仪	Sensor.TYPE_GYROSCOPE	SensorEvent.values[0]	$gyro_x$	rad/s
		SensorEvent.values[1]	$gyro_y$	
		SensorEvent.values[2]	$gyro_z$	

其中，对于加速度计的三个向量，

values[0]：x 轴上的加速度-Gx

values[1]：y 轴上的加速度-Gy

values[2]：z 轴上的加速度-Gz

Gx、Gy、Gz指的是重力加速度在三个方向上的分解量。手表中的传感器测量的是应用于设备的加速度(a)。从概念上讲，加速度计应该是通过使用以下关系测量施加在传感器本身上的力来实现的，m为物体质量：

$$a = -\sum F_s / m \tag{2.1}$$

但是重力总是会影响测得的加速度：

$$a = -g - \sum F / m \tag{2.2}$$

出于这个原因，当设备放置在桌子上并且没有加速时，加速度计读取的数据将会是 $g = 9.81\,m/s^2$。但是当设备处于自由落体状态并因此以重力加速度计g向地面加速下落时，其加速度计读数将会变为 $0\,m/s^2$。因此，实际上加速度计得到的示数是不准确的，存在重力的影响。因此为了测量设备的真实加速度，就必须消除重力对加速度的影响，得到无重力的数据。这可以通过应用高通滤波器来实现。事实上很多设备内置的线性加速度计可以直接得到排除重力影响的三轴加速度，但是由于本次实验用到的手表未内置线性加速度计，我们只能通过后续处理获得排除重力影响的体加速度，实验最后应用到了高通滤波器消除重力对加速度计的影响，具体过程将在第三章进行阐述。

2.2.2 数据处理：卡尔曼滤波器

我们将获取的数据经过卡尔曼滤波器处理后进行输出。卡尔曼滤波器的用处主要是处理传感器的未经处理的数据，消除其中噪声的影响。它能够使用随时间观察到的一系列测量值，包括统计噪声和其他不准确的值进行分析，最终在之前数据基础上，估计未来的数据。这些估计往往是通过估计每个时间范围内变量的联合概率分布产生的，因此比仅基于单个测量值的结果更准确。

卡尔曼滤波器算法总共分为两个阶段。在预测阶段，卡尔曼滤波器会计算得到当前状态变量的估计，以及它们之间的不确定性。一旦滤波器观察到下一次测量的结果（必然会因一些错误而损坏，包括随机噪声），这些估计值就会使用加权平均值更新，并赋予具有更大确定性的估计值更多的权重[28]。该算法是递归的并且仅需要使用当前输入测量值和先前计算的状态及其不确定性矩阵而不需要额外的过去信息。

无控制离散型卡尔曼滤波器的基本方程如下[28]：

系统的状态方程：$x_k = \phi_{k,k-1} \times x_{k-1} + \Gamma_{k-1} w_{k-1}$ (2.3)

系统的测量方程：$Z_k = H_k \times x_k + v_k$ (2.4)

此处，w_{k-1}为过程白噪声，v_k为测量噪声，Γ为噪声驱动矩阵，Z_k为系统测量方程的输出量（即观测量），是可以实际测量的量。

如果此时w_{k-1}和v_k满足

$$\begin{aligned} &E[w_k] = 0，Cov[w_k, w_j] = Q_k \delta_{kj} \\ &E[v_k] = 0，Cov[v_k, v_j] = Q_k \delta_{kj} \\ &Cov[w_k, v_j] = 0 \end{aligned} \tag{2.5}$$

其中Q_k是位过程噪声的协方差矩阵，表示w向量元素之间的相关关系，其为非负定阵；R_k为测量噪声的协方差，表示v向量元素之间的相关关系，其为正定阵。

此时无控制离散型卡尔曼滤波的基本方程为[39]：

（1）对 k 时刻状态的预测方程：

$$\hat{x}_{k/k-1} = \phi_{k,k-1} \times x_{k-1} \tag{2.6}$$

（2）对 k 时刻预测的均方误差的预测方程：

$$\hat{P}_{k/k-1} = \phi_{k,k-1} \times P_{k-1} \times \phi_{k,k-1}^{T} + \Gamma_{k-1} Q_{k-1} \Gamma_{k-1}^{T} \tag{2.7}$$

（3）滤波增益方程（权重）：

$$K_k = \hat{P}_{k/k-1} \times H_K^T [H_k \times \hat{P}_{k/k-1} \times H_K^T + R_k]^{-1}{}_{k,k-1} \tag{2.8}$$

（4）滤波估计方程（k 时刻的最优估计值）：

$$x_k = \hat{x}_{k/k-1} + K_k [Z_k - H_k \times \hat{x}_{k/k-1}] \tag{2.9}$$

（5）滤波均方误差更新矩阵（k 时刻最优均方误差）：

$$P_k = [I - K_k \times H_k] \times \hat{P}_{k/k-1} \tag{2.10}$$

公式中的变量定义如表 2.2 所示：

表 2.2：卡尔曼滤波方程变量及定义

变量	定义
x_k	k时刻的后验状态估计值，即更新后的结果，也叫最优估计
$\hat{x}_{k/k-1}$	k时刻的先验状态估计值，即根据上一时刻（$k-1$时刻）的最优估计预测的k时刻的结果
P_k	k时刻的后验估计协方差，表示状态的不确定度
$\hat{P}_{k/k-1}$	k时刻的先验估计协方差（$\hat{x}_{k/k-1}$的协方差），是滤波的中间计算结果。
H	是状态变量到测量（观测）的转换矩阵，表示将状态和观测连接起来的关系，是滤波的前提条件之一
Z_k	测量值（观测值），是滤波的输入
K_k	滤波增益矩阵，即卡尔曼系数
$\phi_{k,k-1}$	状态转移矩阵
$Z_k - H_k \times \hat{x}_{k/k-1}$	实际观测和预测观测的残差

式中K_k和$Z_k - H_k \times \hat{x}_{k/k-1}$一起修正预测值，从而得到后验值。

目前，我们主要采用加速度计和陀螺仪两个传感器的信息，它们一个读取速度，另一个读取位置。每个传感器都提供了当时传感器状态的信息并产生一组读数。由此我们开始应用卡尔曼滤波器过滤这两份数据、去除噪声，使存储的传感器数据更

加精确，我们定义以下变量：

表 2.3：卡尔曼滤波器变量定义

变量	定义
mX0	预测的状态
mP0	预测的协方差
mF	实际价值与以前实际价值的比率
mH	测量值与实际值的比值
mQ	测量噪声
mR	环境噪声
mState	当前状态
mCovariance	当前的协方差

根据上面的变量，我们可以列出随着时间变化的状态预测方程：

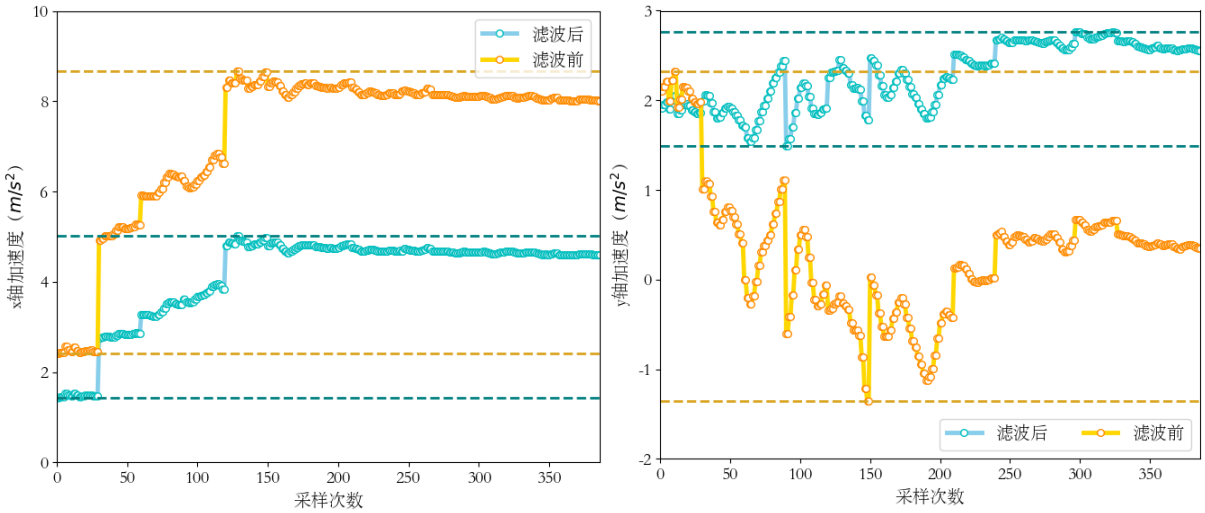
$$\begin{gathered} \mathrm{mX0} = \mathrm{mF} * \mathrm{mState} \\ \mathrm{mP0} = \mathrm{mF} * \mathrm{mCovariance} * \mathrm{mF} + \mathrm{mQ} \end{gathered} \tag{2.11}$$

进而我们可以通过以下公式计算得到修正后的当前状态：

$$\begin{gathered} \mathrm{k} = \mathrm{mH} * \mathrm{mP0}/(\mathrm{mH} * \mathrm{mP0} * \mathrm{mH} + \mathrm{mR} \\ \mathrm{mCovariance} = (1 - \mathrm{k} * \mathrm{mH}) * \mathrm{mP0} \\ \mathrm{mState} = \mathrm{mX0} + \mathrm{k} * (\mathrm{measuredValue} - \mathrm{mH} * \mathrm{mX0}) \end{gathered} \tag{2.12}$$

mState即为我们需要计算的当前修正后的状态，因此我们可以返回该值进行下一步存储。

通过卡尔曼滤波器滤波后的数据相比于原始数据波动更小，可以排除一部分外界噪声的干扰。我们以实验中在经过卡尔曼滤波器前后的 x 轴与 y 轴加速度计数据为例，对比卡尔曼滤波器的效果。如图 2.2 所示，无论是 x 轴还是 y 轴的数据，经过卡尔曼滤波之后，其数据更加平滑，波动幅度更小。加速度在这 400 次采样中的极差明显变小。由此可以看到，卡尔曼滤波器可以有效对我们获取的数据进行处理，去除一部分噪声和其他会影响准确值的因素，使最终获得的数据波动更小，排除干扰。



(a)加速度计 x 轴滤波效果 (b) 加速度计 y 轴滤波效果

图 2.2：卡尔曼滤波前后效果对比

实验中从陀螺仪和加速度计获得的三轴加速度和角速度的数据都将经过卡尔曼滤波器滤波后再进行存储。

2.2.3 基于 FTP 协议的文件存储

系统用到了 FTP 协议对修正后的数据进行传输和存储，在这里我们主要用到了 ftp.FTPClient 和 ftp.FTPReply 包进行文件的存储。

FTP 协议即文件传输协议，是一种标准通信协议，用于将计算机文件从服务器传输到计算机网络上的客户端。FTP 建立在客户端-服务器模型架构之上，在客户端和服务器之间使用单独的控制和数据连接[29]。

在系统中，我们通过地址、端口、密码、用户名、存储路径等信息，将获得的传感器信息记录在.csv 文件中，并以 UTF-8 格式存储，最后通过 FTP 协议存储至手表内部指定的路径。由于实验所用的手表型号 TicWatch Pro3 没有可以直接使用的手表内部文件查看软件，我们通过 NavExplorer 来确认文件是否已经正确存储在手表内部，并通过该软件与电脑连接，使电脑上最终能够得到相应的文件进行下一步数据处理。

2.3 系统展示

整个系统的功能包括了对传感器数据的直接获取、对初始数据用卡尔曼滤波器进行滤波整理、对修正后的数据进行存储，最终能够顺利完成项目的数据收集部分要求。

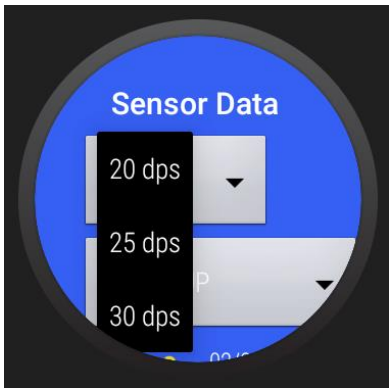
本文将展示该系统在真实的手表和电脑手表模拟器上的展示效果。由于软件未在手表的应用商店上线，因此我们可以用 ADB（Android Debug Bridge，安卓调试桥）连接手表和电脑，使代码在手表上运行。软件通过代码运行在手表内的外观如图 2.3(a)所示。

(a) APP 图标

(b) 软件打开页面

图 2.3：系统图标

打开软件后页面如图 2.3(b)所示，可以选择采样频率和活动标签来收集数据。采样频率分为 20dps、25dps、30dps，分别代表每秒采样 20 次、25 次和 30 次（如图 2.4(a)）。同时可以选择不同数据标签来为自己启动软件中的传感器数据进行自动标注，如 WALK UP、WALK DOWN、SITTING、WALKING 等（如图 2.4(b)），由于本次实验是对人体活动的六个状态进行识别，根据实验内容需要，我们最后设置了 WALKING、WALKING_STAIRS、CIRCLING、SITTING、STANDING、LAYING 六个数据标签，可以直接在收集数据时对数据进行标注，防止混淆。



(a) 采样频率选择

(b) 活动标签选择

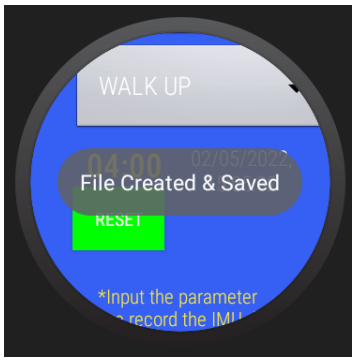
图 2.4：系统操作界面

下滑页面即可点击“RECORD”按钮开始数据采集（如图 2.5(a)和(b)所示）。由于手表容量和性能有限，如果数据采集时间过长可能导致最后生成.csv 文件时手表卡顿，过短则会导致数据时间序列不足，因此我们设定每次数据采集的时间为 4 分钟（时间可根据需要进行更改）。在数据采集过程中，APP 屏幕中将显示 4 分钟倒计时，如图 2.5(c)所示，最终采样完成将会自动显示文件已存储，并且显示重置按钮，如图 2.5(d),单击重置按钮即可回到图 2.5(a)所示界面，继续进行数据采样。

(a) RECORD 按钮

(b) 单击 RECORD 后开始计时

(c)计时采样中



(d) 计时采样完成

图 2.5：数据采集过程

APP 部署在 TicWatch Pro 3 手表上的实际页面显示如图 2.6。

(a)实际页面显示 (b) 实际页面显示 (c) 实际页面显示

图 2.6：APP 在 TicWatch Pro 3 手表上的显示

采集后的数据将以.csv 格式保存在手表中，在文件名中显示采样时间和采样标签，在表格中显示每次数据的采样具体日期、时间、三轴加速度计数据、三轴陀螺仪数据及数据标签，后续的部分实验内容将根据我们所采集到的智能手表传感器数据进行进一步处理和分析。

3 活动识别算法研究

3.1 技术路线

随着深度学习技术的发展，许多新的解决活动识别问题的算法都被提出，人们开始关注到活动识别问题的更多角度。相比于传统的机器学习算法，深度学习网络更加准确，更少依赖人工的数据提取，还可以发现网络中一些隐藏的特征和进行无监督学习，因此我们采用目前热门的深度学习网络对数据进行处理和建立模型。

首先，我们将会选择合适的深度学习网络对收集到的数据集进行训练，并将收集到的数据按照 7:3 的比例划分为训练集和测试集进行研究。在搭建深度学习网络时，可以结合几种前沿的网络，最后将训练得到的模型在验证集上验证其判断准确度，同时也可以通过采用已有的较权威的传感器数据集进行模型精度的验证。同时，本文也会将已经完成的模型在训练集上的训练效果与其他论文中模型的效果进行比较，若精度较低，则利用一些深度学习网络的优化方法进行模型的优化，使模型取得较好的效果。

因此，本文的技术路线如图 3.1 所示，我们会先将传感器得到的数据进行预处理，之后输入深度学习网络进行训练，并通过网络对人不同的活动状态进行识别和分类，最终输出结果，同时与正确的结果进行比较，得到网络的精确度，最后进一步对网络进行调整和修改。

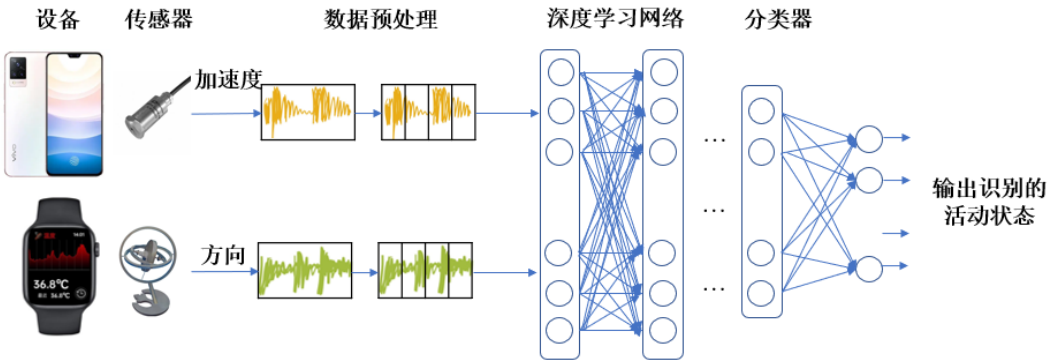


图 3.1：技术路线

3.2 数据集

本次活动识别主要研究的是我们身边的活动，我们在实验中，需要对比模型在智能手机和智能手表传感器数据处理上的效果,因此需要两个数据集来完成所有的工作。

3.2.1 智能手机数据集

由于目前智能手机数据的研究已经非常深入，并且有大量开源的数据集，自己收集数据往往反而会导致数据过于粗糙，不如已有的数据，最终导致数据不精确。因此我们采用目前已有的智能手机数据集进行研究。在这里我们用到的智能手机的数据库是来自 UCI 的 A Public Domain Dataset for Human Activity Recognition Using Smartphones[30]，该数据库中的数据实验是在一组年龄在 19-48 岁的 30 名志愿者中进行的，他们每个人都在腰上佩戴了三星 Galaxy S II 智能手机，并进行了六项活动(走路、上楼、下楼、坐、站、躺)，六种状态的定义如表 3.1 所示。利用其内置的加速度计和陀螺仪，该数据库的研究者们以 50 次/秒的频率对智能手机内置加速度计和陀螺仪的 3 轴线性加速度和 3 轴角速度进行手机，并手动标记数据。他们得到的数据集被随机分成两个集合，其中 70%的志愿者被选为训练数据，30%的志愿者被选为测试数据。对于生成的数据，研究者们通过噪声滤波器对传感器信号进行预处理，然后通过 2.56 秒的固定宽度的窗口进行划分数据和采样，其中数据包括了 50%的重叠，最终使每个数据具有 128 的时间步长。研究者提供的每个样本都有 561 个来自信号分析的线性手工预处理特征（例如，窗口的峰值频率），但我们的研究中只使用了其中 9 个特征：来自加速度计的三轴重力加速度和去除重力因素后估计的体加速度以及来自陀螺仪的三轴角速度，特征及对应变量如表 3.2 所示。这些时域信号(前缀“t”表示时间)是以 50Hz 的恒定速率捕获，并且已经使用中值滤波器和角频率为 20Hz 的三阶巴特沃斯低通滤波器滤除噪声，同时已经全部进行过归一化处理，被限制在$[-1,1]$内。

由于所有数据已经经过基本处理，智能手机的数据集可以直接进行实验。

表 3.1：智能手机数据活动状态

序号	状态	变量名	定义
1	走路	WALKING	用户腰上佩戴手机，自然步行
2	上楼梯	WALKING_UPSTAIRS	用户腰上佩戴手机，正常步行上楼梯
3	下楼梯	WALKING_DOWNSTAIRS	用户腰上佩戴手机，正常步行下楼梯
4	坐	SITTING	用户腰上佩戴手机坐在椅子上
5	站	STANDING	用户腰上佩戴手机静止站立

6	躺	LAYING	用户腰上佩戴手机躺在床上

表 3.2：数据变量及数据内容

变量名	定义
$body_acc_x, body_acc_y, body_acc_z$	去除重力因素的体加速度
$gyro_x, gyro_y, gyro_z$	从陀螺仪获得的三轴角速度
acc_x, acc_y, acc_z	从加速度计获得的三轴加速度

3.2.2 智能手表数据集

由于目前网络上有关智能手表的数据集较少，本文将自行收集数据构建智能手表的数据集，具体收集过程详见第二章。由于自己设计的数据收集 APP 暂无法在谷歌商店上线，因此无法直接安装到手表上，手表只能在电脑附近，通过联网进行数据的手机和传输。参考智能手机数据集的六个活动分类：走路、上楼、下楼、坐、站、躺，由于条件限制，上楼和下楼无法持续收集数据，因此我们将上楼、下楼合并为上下楼类，并且新增一类原地转圈活动的数据。新的六种活动定义如表 3.3 所示。

表 3.3：智能手表数据活动状态

序号	状态	变量名	定义
1	走路	WALKING	用户左手手腕佩戴手表，自然步行
2	上下楼梯	WALKING_STAIRS	用户左手手腕佩戴手表，正常步行循环上下楼梯
3	原地转圈	CIRCLING	用户左手手腕佩戴手表，在原地顺时针转圈
4	坐	SITTING	用户左手手腕佩戴手表坐在椅子上
5	站	STANDING	用户左手手腕佩戴手表静止站立
6	躺	LAYING	用户左手手腕佩戴手表躺在床上

对于手表收集到的已经经过卡尔曼滤波器处理的原始数据，仅包含原始的三轴加速度数据和角速度数据，由于实验采用的手表型号没有内置线性加速度计，不可以直接获得过滤重力的加速度数据，将首先使用高通滤波器根据已有数据估算得到不受重力影响的体加速度。

使用高通滤波器滤除重力的公式如下[31]：

$$gyro_i = \alpha * gyro_i + (1 - \alpha) * acc_i \tag{3.1}$$

$$body_acc_i = acc_i - gyro_i$$

其中α=$t/(t+T)$，t是低通滤波器的时间常数，T 是采样时间，i代表x,y,z三轴信号。

由此我们可以得到体加速度值，处理过后数据变量及其内容仍旧如表 3.2 所示。最后将得到的数据根据 7:3 比例区分为训练集和测试集来进行后续的研究。

综合这两个数据集的结果，我们在正常对比不同模型的效果之外，还可以对比同一模型在不同数据集上的效果，使我们的结论更具有普适性和准确性。

3.3 数据处理

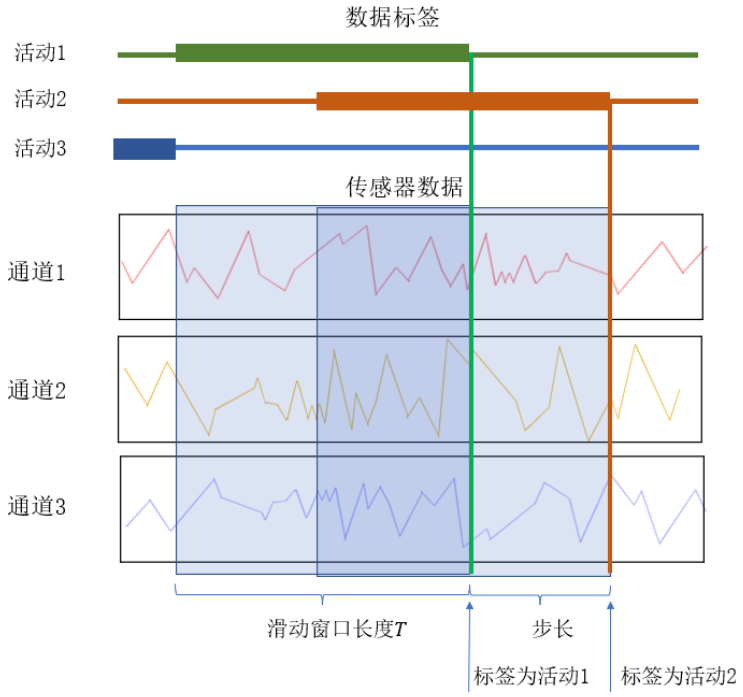


图 3.2：滑动窗口标记活动标签

在将数据输入运行主干算法之前，还需要对数据进行处理。当数据缺失时，我们会将数据相加，然后归一化为均值为 0 和标准差为 0.5 的数据。同时数据需要重塑以适应本文所设计的网络，由于数据序列本身长短不同，但是实验需要我们对窗口长度进行规范，最后统一为长度为 128 的样本。因此我们先用滑动窗口对数据集进行了处理，具体过程如图 3.2 所示。对于 3 个通道的数据集，其可能代表着不同的运动状态，我们在图中以可能的三个活动状态为例进行说明。在每一个时间序列类型的数据上，由一个窗口长度为T的序列不断在数据集上运行获得样本。$1/2T$将会作为步长移动以形成下一个样本，每一次移动保证 50%的重叠，这在训练过程中增加了一些冗余。重复上述操作，然后我们可以得到适合训练的数据集。接着

对于每一个样本，我们选择每个样本即滑动窗口的最后一刻的数据标签作为整个样本分类的标签。因此，即使一个数据在时间序列中发生变化，也只有最后一个数据发生变化时会影响整个被用于进行训练和测试的序列活动分类。而其中 50% 的重复将会被用于学习一些标签变化，从而反映出数据的意图以及人当时可能的活动和行为。最终，通过该处理，每一个不同长度序列活动都被分为多个长度为 128 的一个活动进行存储，数据集 $sequenceNum \times Channels$的形状将会被转换为具有一定重复的 $sampleNum \times windowSize \times Channels$形状，最终两个不同数据集的大小和参数将会在模型结果部分作进一步描述。

3.4 基于深度学习的活动识别算法

3.4.1 基础的 LSTM 结构

活动识别即 HAR 问题，因为状态预测会受到之前的状态的影响，因而可以被看做一个时间序列处理的问题。LSTM 就是知名的用于处理时间序列问题的算法。与传统算法不同，LSTM 可以在时间上捕捉数据中的关系和维度，且不会像一维卷积神经网络 (CNN) 那样将时间步长混合在一起。随着更多数据的出现，LSTM 架构可以提供出色的性能和许多潜在的应用。

LSTM (Long short-term memory)算法最早是由 Juergen Schmidhuber 在 1997 年提出的，它是循环神经网络的变体并且具有特殊的性质使其能够在时间序列问题的处理上具备比其更好的性能。由于 2012 年深度学习的兴起，LSTM 经过若干科学家补充和应用，再次焕发了新的生机。与 CNN、受限玻尔兹曼机 (RBM) 和自动编码器 (AE) 等其他网络相比，LSTM 能够使梯度能够轻松地流经时间，因此其特别擅长解决涉及时间序列的问题[32]。一个常见的 LSTM 单元由一个单元、一个输入门、一个输出门和一个遗忘门组成。单元能够在某个时间间隔内记住流过的数据值，而剩下的三个门被用来调节信息的流入和流出。如图 3.3 所示即为基础 LSTM 的结构，图中每个方框表示一个神经网络层，由权值、偏置以及激活函数组成，每个圆圈表示元素级别操作，箭头表示向量流向，相交的箭头表示向量的拼接，分叉的箭头表示向量的复制。每个方框内的结构都是基本相似的。其最核心部分是在单元中的绿色传送带部分，这一部分一般叫做单元状态（cell state），并且存在于 LSTM 的每个单元中[33]。在图中网络的传递过程可以描述为：输入量与隐藏层变量通过矩阵变换到隐藏层。然后隐藏层的输出通过一个激活函数进行计算，最终得到最后的输出值。

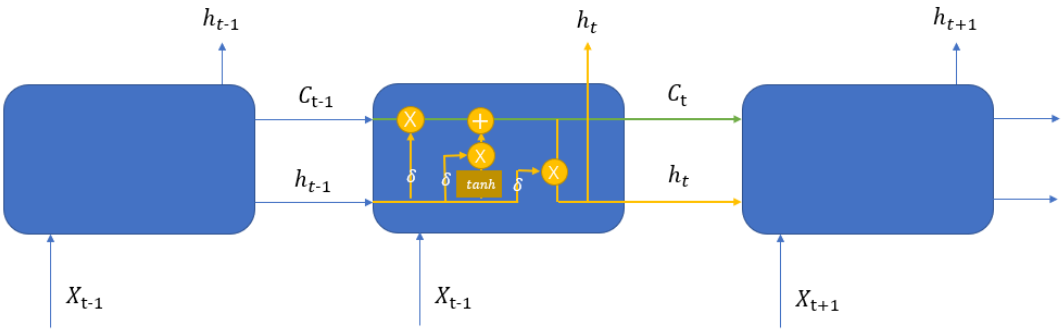


图 3.3：LSTM 结构

如果我们定义输入变量为$\{X_0, X_1, X_2, \dots, X_n\}$为输入变量，$\{h_0, h_1, h_2, \dots, h_n\}$为隐藏层的输出变量，$\{y_0, y_1, y_2, \dots, y_n\}$为 LSTM 的输出变量，$U$、$W$、$V$分别表示从输入层到隐藏层、从隐藏层到隐藏层、从隐藏层到输出层的权重度量，则我们可以列出最后变量表示的式子为：

$$h_i = \begin{cases} g\left(UX_i + b_i^h\right), i = 0 \\ g\left(UX_i + Wh_i + b_i^h\right), i = 1,2, \dots \end{cases} \tag{3.2}$$

$$y_i = g\left(Vh_i + b_i^y\right), i = 0,1, \dots \tag{3.3}$$

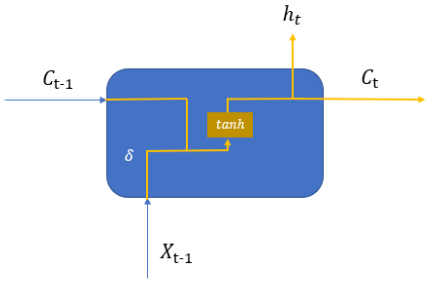


图 3.4：基础 RNN 单元结构

我们将 LSTM 结构与 RNN 结构(图 3.4)进行对比可以发现，虽然 RNN 也可以根据过去的信息估计当前时间的输出。然而，Bengio [30]发现，由于梯度消失和爆炸的问题，RNN 只能在短时间内记住信息。当使用深度网络的反向传播时，如果不采取允许梯度深入流动的预防措施，梯度将迅速消失。因此相比于 RNN，LSTM 具有更加有效的预测时间序列的效果。在这里，LSTM 首先需要遗忘过去的旧信息，因此会涉及到遗忘门。而下一步将由输入门来确定哪些新信息需要保存在内存中。由此，就可以将旧的单元状态C_{t-1}更新为新的单元状态C_t。最后会决定哪些信息将通过输出门输出到上面的层，这就是 LSTM 的基本工作过程。

3.4.2 双向 LSTM 结构

LSTM 虽然在普通的 RNN 模型上做了一定的修改，增加了很多门调控输入结构，但是其每个单元的状态矢量输入下一时刻的单元这一过程是单方向的。单元只能根据以前的信息来预测当前状态。很明显，如果该过程只在一个方向上运行，某些重要信息可能无法被正确捕获。因此，在 2015 年研究者们发表了一篇有关双向 LSTM 的论文[34]，其改进在于是 LSTM 的结构变为双向，且使其当前的输出不仅与先前的信息有关，还与后续的信息有关。在文中双向 LSTM 即 Bidirection-LSTM（简称为 Bidir-LSTM 网络）主要被利用于自然语言处理的上下文标注和预测领域，可以看出，其对有时间序列特征的数据的预测也是合适的。具体来说，双向 LSTM 将其推广为两层的 LSTM，两层的状态输入方向相反，新加的一层一步一步将状态向序列的上一元素传递。两层 LSTM 的隐藏层状态分别向前和向后传播，最后结合两层 LSTM 的输出得到标注结果。图 3.5 所示即为双向 LSTM 的结构，其中红线代表了隐藏层向前传播的状态。图中的隐藏层中有前向序列$\vec{h}$和后向序列$\overleftarrow{h}$，此时，隐藏层和输入层可以定义如下：

$$\vec{h}_t = g\left(U_{\vec{h}} x_t + W_{\vec{h}} \vec{h}_{t-1} + b_{\vec{h}}\right) \tag{3.4}$$

$$\overleftarrow{h}_t = g\left(U_{\overleftarrow{h}} x_t + W_{\overleftarrow{h}} \overleftarrow{h}_{t-1} + b_{\overleftarrow{h}}\right) \tag{3.5}$$

$$y_t = g\left(V_{\vec{h}} \vec{h}_t + V_{\overleftarrow{h}} \overleftarrow{h}_{t-1} + b_y\right) \tag{3.6}$$

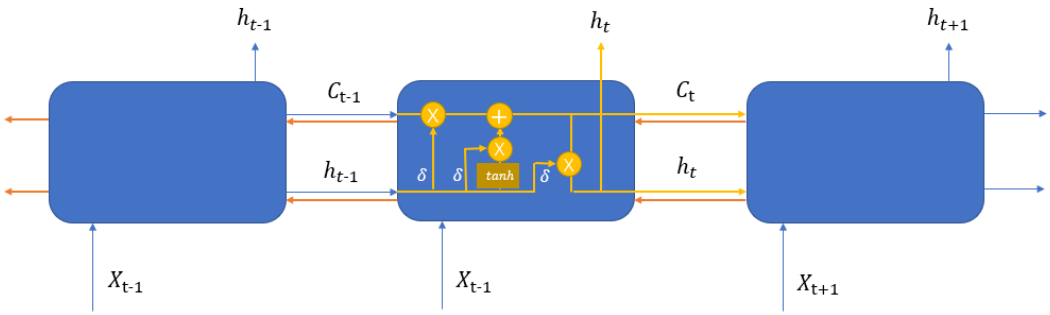


图 3.5：双向 LSTM 结构

3.4.3 残差网络

残差网络是在 2015 年第一次被提出的，并在图像分类和物体识别方向大放异彩，取得了良好的效果，因此得到了学者们的注意。残差网络的提出主要是为了解决超深度 CNN 网络训练问题，尤其是梯度消失/爆炸问题。很多网络在训练时会遇到阻碍收敛问题，标准初始化和正则化（SGD+反向传播）可以较好地解决这系列

问题。但同时普通网络也会面对退化问题：网络不是越深越好，随着网络深度的增加，准确度上升到一定值后会迅速退化。而残差网络可以较好地解决这些问题，对于普通网络，可以向其中插入快捷连接(shortcut)来实现残差学习。快捷连接不引入额外的参数和复杂度，这不仅在实践中很有吸引力，而且便于比较普通网络和残差网络，具有相同数量的参数、深度、宽度和计算成本。

残差连接通过前面的层到下一层的特征的馈送（通过求和），可以轻松训练具有大量（甚至数千）层的网络，而不会增加训练错误百分比，可以有助于使用恒等映射解决梯度消失问题。总体而言，其能够加快训练速度的深层网络，增加网络深度而不是扩大网络，导致更少的额外参数，并能够减少梯度消失问题的影响，最后获得更高的网络性能准确度。

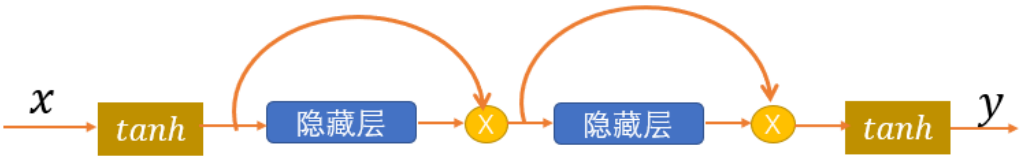


图 3.6：残差网络结构

残差网络的具体结构如图 3.6 所示，下层部分直接通过高速网络传输到上层，增加了信息流动的自由度。包含跳跃连接的高速网络结构可以在输出之前取得不同层数的补充，我们将层数用n表示。当$n=0$时，将没有残差连接。此时网络就会如同普通的的深度堆叠 LSTM 层一样。当我们给网络添加了残差结构，则隐藏层$i(i=1,2,\dots,L)$的输出可以用如下式子表示：

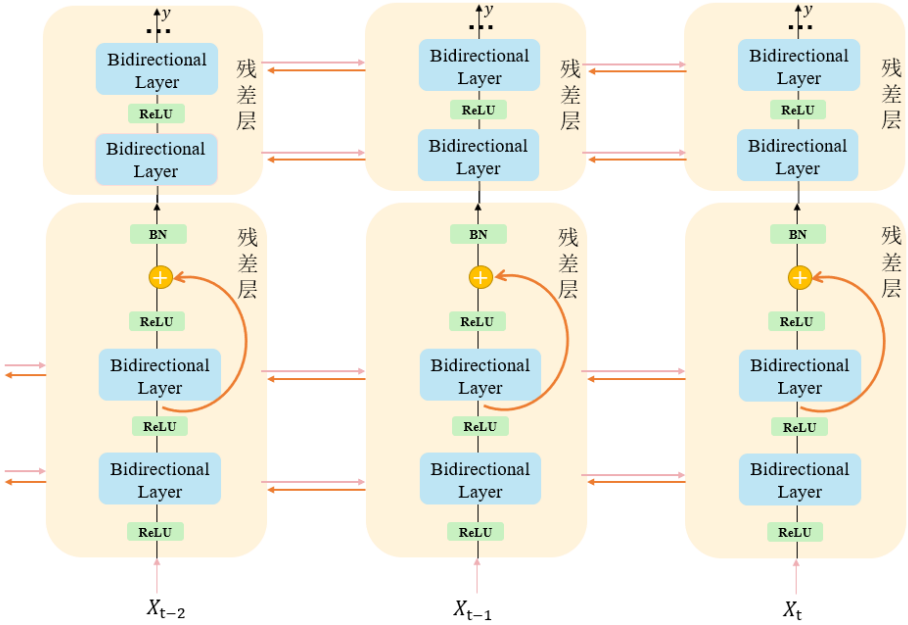
$$h_1=\sigma(W_1x+b_1), i=1 \tag{3.7}$$

$$h_i=\sigma(W_ih_{i-1}+b_i)+h_{i-2}, i=2,3,\dots,L-1 \tag{3.8}$$

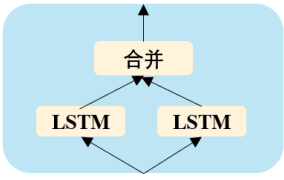
$$y=\sigma\left(W_yh_i+b_y\right)+h_{i-1}, i=L \tag{3.9}$$

最终构造的网络即 Res-LSTM。

3.4.4 深度残差的双向 LSTM 网络



(a)



(b)

图 3.7：深度残差的双向 LSTM 网络结构

基于基础的 LSTM 网络，结合了残差网络和双向 LSTM 网络，最终可以构造得到 DeepHAR 网络。与本文中模型类似结构如图 3.7 所示，图中的网络具有总共 12 个 LSTM 单元，实验中的我们所使用的网络是 3×3 架构，共具有 18 个 LSTM 单元。图中黄色部分即代表了残差层，每个残差层的结构都是相似的，正如第二排每一个残差层的结构所示。残差层内部的每一个双向层（用蓝色表示）即 Bidirectional Layer 的内部结构如图 3.7(b)所示。

由于传感器的数据受到前后时间的数据影响，作为一个时间序列数据集，其采用 LSTM 进行处理，可以取得良好的效果，并且保留其在时间维度上的特征。目前 ResNet 得到了广泛重视，残差网络在 CNN 网络上的应用大为流行，但是残差网络在 LSTM 也同样有效。因此我们仿照 ResNet，将残差连接添加到了双向 LSTM 网络上。

深度残差的双向 LSTM 神经网络在可以访问的数据点数量方面受到了一定的

限制，因为我们设定数据集单个为 128 个时间步长，因此模型在进行预测时只会访问 128 个时间步长。LSTM 模型在深度加深时，下一个前向或者后向都将“提前”看到另一个通道的输出，因为双向 LSTM 的后向通道在连接之前反转了输入和输出，因此，双向 LSTM 具有与基础 LSTM 相同的输入和输出形状，但是通过在给定的时间步长不断向后传递，它可以提前访问更多信息，具有更好的效果。

我们最终构建的网络将会在时间维度和模型深度两个方面进行流动，除了输入层和输出层外，有 2 个隐藏层内部有残差连接，因此我们将其称为残差层。此外，每个残差层包含 2 个双向层。在我们的网络中，活动函数与 ReLU 是统一的，因为它在对抗梯度消失方面总是优于深度网络的 tanh。尽管输出是给定时间窗口 T 的向量，但时间维度已被神经网络处理过了，也就是说，我们只需要输出的最后一个元素，可以丢弃其他元素。因此，我们可以仅应用来自上一个时间步长的预测梯度，而其中一个 LSTM 单元即双向 LSTM 传递中最上层的后向 LSTM 单元可以被丢弃。此外，在训练过程中我们的模型也会对训练数据集进行打乱重置，对于每个新预测，神经网络的状态都会在每个新窗口处重置。

梯度消失是深度网络的普遍问题，为了解决这个问题，我们向模型中加入了残差层、双向层和堆叠层（即最终的 DeepHAR 网络）来解决这个问题。由于在使用反向传播时，一些底层网络难以得到优化，因此我们将结合每个高速连接层顶部的变量进行批标准化，使残差连接充当梯度的高速连接，从而防止隐藏层特征空间中的限制过于复杂，避免在测试时出现异常值，防止过度拟合。

此外，我们也可以针对正则化的问题优化大型网络，例如添加 L2 权重衰减范数和 $dropout$参数，这两个参数的添加有助于模型训练和防止过拟合。如果不使用正则化，则会导致测试集上的过度拟合。我们在设计网络的时候，也充分考虑了网络的复杂性，因为只有在与正则化相反的情况下，复杂的网络具有较好的性能，否则太多的层数和每层的单元反而会增加计算复杂度并浪费计算资源。事实上当模型层数和单元数达到一定规模时，识别准确率将保持在一定规模而不是增加，因此我们需要选择最恰当的层数，在保证准确度的同时避免浪费资源，使网络可以增加深度正则化，以避免过拟合。

网络中添加的重要参数如下：

$L2$范数：在实验中我们发现，网络在测试集的准确度低于在训练集的准确度，这很有可能是发生了过拟合，并且在实验中由于超参数过多，正则化超参数的平衡变得很困难，正则化就是通过对模型的参数设定一个先验来防止过拟合。因此我们引入了$L2$范数，其将权值向量w中各个元素的平方和然后再求平方根，通常表示为$\|w_i\|_2$，即x向量各个元素平方和的 1/2 次方，根据$L2$范数的特性，它会使得权值减小，即平滑权值，一定程度上也能和$L1$一样起到简化模型、加速训练的作用。其也

会让特征的权重不过大，使得特征的权重比较平均，同时可防止模型过拟合，所以我们添加了权重衰减的 $L2$ 范数到损失函数中。

***dropout*参数：**模型深度轴的每一层之间还添加了$dropout$参数，这是另外一个超参数。$dropout$指隐藏层输出的部分向量在每个训练层中会以一定的概率变为 0，而其他值相应地按比例放大以保持向量的相同几何范数值。模型中不工作的节点可以被认为是暂时不在网络中的死节点（或神经元），这意味着这些节点背后的权重和偏差在每一层的训练过程中暂时既不学习也不有助于预测，一直保持不变。本模型中的$dropout$添加在输入和每个堆叠层之后(但不添加在剩余层之间)。

最大梯度步长范数：为了避免在实验过程中模型准确率突然趋于平稳，我们通过梯度裁剪[35]添加了最大梯度步长范数，它能有效地权重控制在一定范围之内，其算法伪代码如下：

$g \leftarrow \partial C / \partial W$
if $\|g\| \geq clip_gradient$ then
    $g \leftarrow clip_gradient \times g / \|g\|$
end if

其中我们假设阈值为$clip_gradient$，其是一个超参数，接着我们通过将梯度的单位向量乘以阈值来裁剪梯度。伪代码中g是梯度，而$\|g\|$是g的范数。由于$g/\|g\|$是一个单位向量，重新缩放后新的g将具有等于$clip_gradient$的范数。请注意，如果$\|g\| < clip_gradient$，那么我们不需要做任何事情。通过该梯度裁剪可以确保梯度向量g的范数最多等于$clip_gradient$。总体而言添加该阈值梯度有助于使模型在训练期间不超过权重更新，防止其在权重空间中发生突然的下降。

最后模型变量的输出进行了批处理，即归一化处理。每一层都会通过均值和方差进行简单的归一化，使得整个层数据的均值为 0，标准差为 1，再用一个较大的重新缩放因子乘以所有数据，并补充上比较大的偏差，然后以线性方式对结果进行归一化和偏移。每一层的缩放乘数 α 和偏移参数 β 都通过学习以自定义方式重新缩放和修改输入，并且 β 可以初始化为 1，该公式定义为：

$$\hat{x}^{(k)} = \frac{x^{(k)} - E\left[x^{(k)}\right]}{\sqrt{Var[x^{(k)}]}} \tag{3.10}$$

$$y^{(k)} = \alpha^{(k)}\hat{x}^{(k)} + \beta^{(k)} \tag{3.11}$$

公式中的$x^{(k)}$表示了变量向量中第 k 个变量，$y^{(k)}$则表示了$x^{(k)}$归一化处理后的值。

总的来说，在构建了基本的 LSTM 模型后，我们向模型中添加了残差层、堆叠层等元素，并且为了提供更好的效果，我们使用$L2$范数和$dropout$机制来防止过拟

合，同时还使用了使用梯度裁剪和批处理来防止梯度消失，最终构造的模型可以在两个数据集上获得良好的效果。

3.4.5 超参数设定方法

模型超参数是模型外部的模型特征，其值与数据本身无关，而是在之前就已经设定。因此必须在学习过程开始之前设置超参数的值。算法网络结构中的超参数将会在很大程度上影响网络的效果，但是超参数由于选择范围过大，因此其选择往往会成为一种难题。在这里我们选用了网格搜索算法进行超参数的选定。

网格搜索是一种尝试计算超参数最佳值的调整技术。这是对模型的特定参数值执行的详尽搜索。通过网格搜索可以节省我们的时间、精力和资源。其主要是通过一个步长在给定范围内遍历所有可能的参数实现的。算法最终会确定一个准确度最高的参数，实际操作类似穷举算法的过程。

当网格搜索的超参数的数量增长时，网格搜索的计算复杂度会呈现指数增长，因此不适宜于超参数过多的情况。我们的实验超参数数量较少，因此网格搜索方法适合我们寻找超参数。我们将会在实验中列出每个参数的超参数值域，具体值域将在下一章进行说明。然后通过把这些超参数进行笛卡尔积排列组合输入网络，然后使用网格搜索算法尝试每组超参数训练模型并挑选验证集误差最小的超参数组合，从而改进神经网络，并不断重复这一过程。

4 实验与结果

实验主要是在一台具有 8 GB RAM 的 i7 CPU 的 Windows 电脑上进行的。

4.1 超参数选择

我们在本文中考虑的超参数是通过网格搜索策略进行选择的。为了减少选择空间，我们让所有隐藏层共享相同数量的单元、相同的学习率和相同的噪声水平[2]。在本部分我们将列出不同网络用到的超参数选择范围，由于普通 LSTM 网络和我们优化过后的网络性能不相同，因此不同网络的选择范围不同。同时，由于有学者已经利用基础 LSTM 研究了活动识别，简单 LSTM 网络的多数参数直接根据其研究进行选定。但是对我们的优化网络，我们都通过网格搜索的方法寻找最优超参数，网络超参数选择范围如表 4.1 所示。

表 4.1：超参数范围

变量	名称	简单 LSTM 选择范围	优化网络选择范围
$learning_rate$	学习率	0.0025	$[0.01, 0.007, 0.001, 0.0007, 0.0001]$
$lambda_loss$	$L2$ loss	0.0015	$[0.001, 0.005, 0.01]$
$training_epochs$	epoch 数量	$[100, 200, 250, 300]$	$[100, 200, 250, 300]$
$batch_size$	批处理参数	[1500, 2000]	$[100, 200, 250, 300]$
$clip_gradients$	梯度裁剪	--	[10,15]
$keep_prob_for_dropout$	$Dropout$	--	0.85

4.2 算法在智能手表数据集上的效果

4.2.1 数据集分析

我们首先验证算法在自己构建的智能手表数据集中的效果，由于该数据集由自己构建，因此需要对数据进行较多的预处理，从而使数据集可以达到模型需要的效果。

首先将对输入的数据集基本情况进行分析如下：

输入传感器的测试数据共有 451 条，训练数据共 1089 条，训练集数量与测试集数量的比例约为 7:3。由于数据是由我们自己采集的，因此相对公开数据集较为粗糙，在不同动作之间的转换可能会存在相似，比如在走楼梯间隙可能人正在走路，

但是也被归到了走楼梯类别。同时每条自己采集的数据长度不一，需要首先利用滑动窗口协议进行裁剪，裁剪后每条数据的长度为 128，即包含 128 个采样点，对应训练集有 1089 条数据标签，数据标签共 6 种。测试数据传感器数据的平均值为-0.67311484，数据标签方差为 2.982389。

我们可以大致对数据集的六种数据集进行基本的分析，六个类型数据集的数据量如图 4.1 所示：

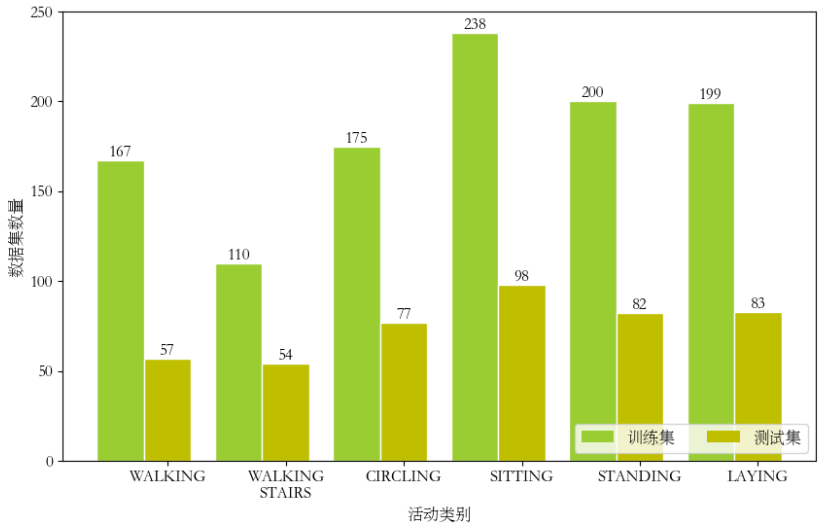
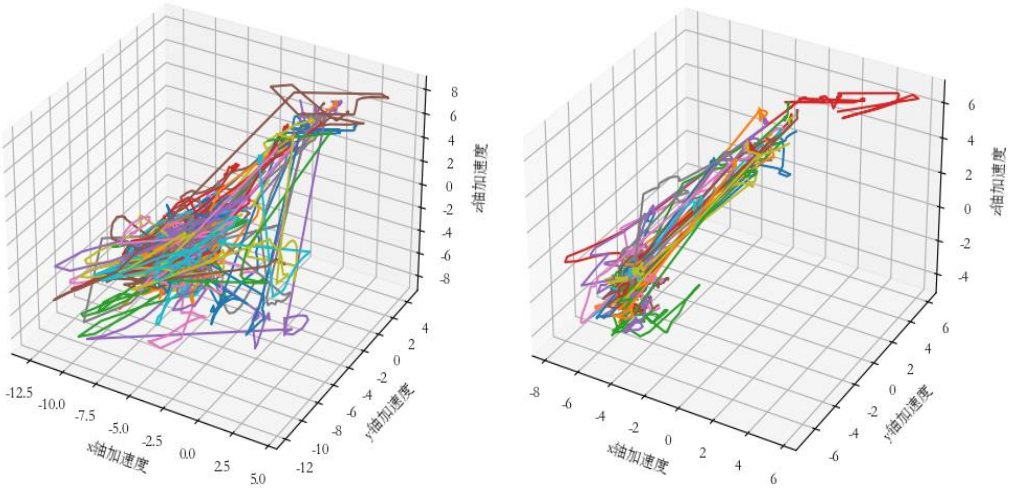


图 4.1：智能手表数据集数据量



(a) walking 状态三轴加速度 (b) standing 状态三轴加速度

图 4.2：walking 和 standing 状态的三轴加速度特征

我们对收集到的不同类型数据进行分析，此处以三轴加速度计获得的数据为例，我们选取了第一类 walking 和第五类 standing 状态的 100 条长度为 128 的三轴加速

度进行比较，如图 4.2(a)(b)所示，可以看到两种状态的加速度变化存在较大的区别，因此数据收集是有效的，可以通过深度学习时间序列的方法对其特征进行学习最后加以区分。

由于该数据集全部由两位人员志愿收集，同时数据收集时活动范围受限，因此可以想见数据集应该存在一定的偏差，为了验证数据集和模型的准确度，我们在最后的模型比较部分会引入他人已完成配置的模型来进行详细分析。

4.2.2 基础的 LSTM 算法

我们首先检验了基础 LSTM 在智能手表传感器数据集上的效果，为了研究初始模型的效果，我们只对基础 LSTM 添加了$L2$ loss 来防止神经网络对数据进行过度拟合。

我们利用 Tensorflow 定义了两个堆叠的 LSTM 单元(两个循环层)，并且根据已有的论文参数来配置我们的模型[32]，由于已有论文的数据集都是运用较为完善的公开数据集，而本次我们的数据集由自己收集，相对较为粗糙，因此需要我们对参数进行调整，尤其是批处理参数，当参数为 1500 的时候训练准确度为 70%，但是在参数调整为 2000 后，准确度可以达到 80%，基于此我们选取了以下较为合适的模型参数：

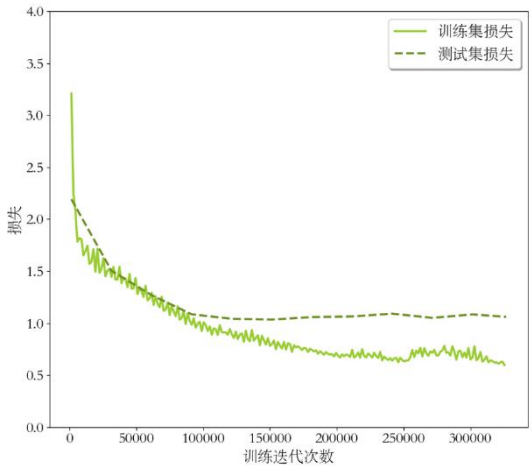
学习率：$learning_rate = 0.0025$

$L2$ loss：$lambda_loss = 0.0015$

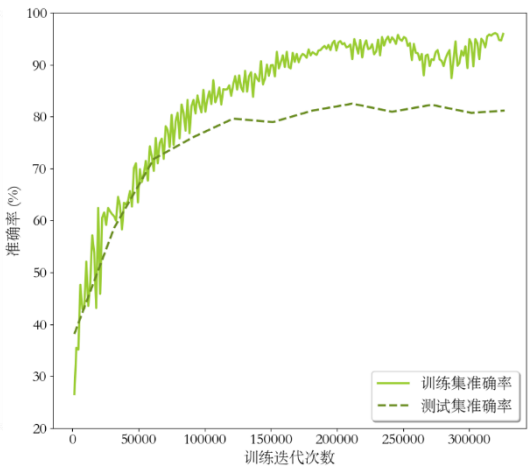
训练迭代次数：$training_iter = training_data_count * 300$

批处理参数：$batch_size = 2000$

训练前利用滑动窗口对每个数据采样，在滑动窗口移动后共生成了 1089 个训练系列，每个系列之间有 50%的重叠，每个系列的时间步长均为 128，每个时间步长有 9 个输入变量，隐藏层数为 32。



(a)模型损失



(b)模型准确率变化

图 4.3：智能手表数据集在基础 LSTM 的训练过程

由于基础 LSTM 的网络效果较差，因此我们需要训练大量次数才能达到相对满意的效果，设置的训练迭代次数为 326700 次，基本在 150000 次左右模型在测试集上的准确率逐渐稳定下来，达到了 80%左右，并且在其左右震荡，如图 4.3 所示。模型在测试集上的损失在迭代为 100000 次逐渐稳定，而在训练集上则持续下降。同时，训练集损失要低于在测试集时的损失，而准确度则高于在测试集上的准确率。我们通过准确度、召回率、$F1\ score$来评估模型的效果。$F1$分数（$F1\ Score$），是统计学中用来衡量二分类模型精确度的一种指标,它同时兼顾了分类模型的精确率和召回率。在计算前我们需要明确以下变量：

$TP(True\ Positive)$：预测答案正确；

$FP(False\ Positive)$：错将其他类预测为本类；

$FN(False\ Negative)$：本类标签预测为其他类标；

$Precision$：精准度/查准率，指被分类器判定正例中的正样本的比重；

$Recall$：召回率/查全率，指被预测为正例的占总的正例的比重；

$Accuracy$：准确率，代表分类器对整个样本判断正确的比重。

则可以计算得到：

$$precision = \frac{TP}{TP + FP} \tag{4.1}$$

$$recall = \frac{TP}{TP + FN} \tag{4.2}$$

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{4.3}$$

计算每个类别下的$F1\ score$，计算方式如下：

$$f1_k = 2 \times \frac{precision_k \times recall_k}{precision_k + recall_k} \tag{4.4}$$

通过对各个类别下的$F1 - score$求均值，得到最后的评测结果如下：

$$score = (\frac{1}{n}\sum f1_k)^2 \tag{4.5}$$

根据以上公式，这次实验最终模型的精确度达到了 81.25383%，召回率为 81.15299%，最终获得的$F1\ Score$为 81.17467，由于数据集的干扰较多，模型达到的精度，已经超出了我们的预期，但是与 90%的准确率还是存在一定差距。模型最后的批损失 Batch Loss 约为 1.06008，还是相对偏高了。模型准确度 Accuracy 约为 81.15299%，因此我们将采用更加优化后的模型对数据进行训练并比较最终的效果。

我们还利用了混淆矩阵展示模型的效果，混淆矩阵是可用于测量机器学习算法

的性能的矩阵。其每一行代表一个实际类的实例，每一列代表一个预测类的实例，通过混淆矩阵，我们可以较为清晰地知道模型每一类的预测效果，并和实际结果进行对比。

初始的混淆矩阵值如下式所示：

$$\begin{bmatrix} 52 & 1 & 4 & 0 & 0 & 0 \\ 31 & 20 & 2 & 0 & 0 & 1 \\ 9 & 0 & 65 & 0 & 3 & 0 \\ 0 & 0 & 0 & 75 & 0 & 23 \\ 0 & 0 & 4 & 1 & 77 & 0 \\ 0 & 0 & 0 & 8 & 0 & 75 \end{bmatrix} \tag{4.6}$$

矩阵中从左到右、从上到下的每一类依次为走路、上下楼梯、原地转圈、坐、站、躺。我们将根据该原始矩阵进行归一化处理：使每一行的数据总和为 1，每个元素显示其占一行总样例数的比例。我们可以得到混淆矩阵如下：

$$\begin{bmatrix} 0.61403 & 0.28070 & 0.10526 & 0 & 0 & 0 \\ 0.33333 & 0.62962 & 0.03703 & 0 & 0 & 0.01851 \\ 0.09090 & 0 & 0.88311 & 0 & 0.02597 & 0 \\ 0 & 0 & 0 & 0.83673 & 0 & 0.16326 \\ 0 & 0 & 0.04878 & 0.04878 & 0.90243 & 0 \\ 0 & 0 & 0 & 0.09638 & 0 & 0.90361 \end{bmatrix} \tag{4.7}$$

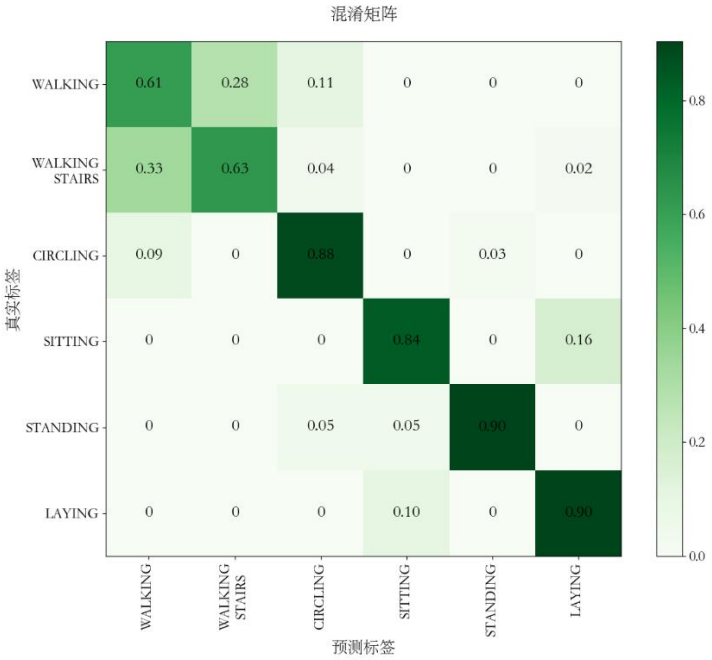


图 4.4：智能手表数据集在基础 LSTM 的混淆矩阵

利用归一化后的矩阵，我们展示了混淆矩阵的可视化效果如图 4.4 所示。可以

发现，基础 LSTM 网络的分类效果还是相对较差，正确标签的分布不是很集中，尤其是 walking 和 walking stairs 两类数据，经常出现模型混淆的情况,其预测正确的准确率都只有 63%左右，因此模型在这两类的辨析能力较差。第 3、4、5、6 类活动相对模型预测分类的准确度较高，但还是有出现混淆的情况。算法整体性能不是很高，因此我们考虑通过优化后的算法对数据进行分类，并将通过优化后的算法进一步验证某些类数据的混淆是否仍然存在。

4.2.3 优化算法

由于基础的 LSTM 性能较差，我们继续尝试优化算法，根据 3.2 中的算法内容，我们分别向基础的 LSTM 添加了堆叠层变为了双向 LSTM 即 Bidir-LSTM 网络、添加了残差网络变为 Res-LSTM 网络，并在最后将残差连接添加到了双向 LSTM 网络上构建了最后的 DeepHAR 网络来查看效果。对于优化后的三个模型，我们确定了模型的参数设置如下：

学习率：$learning_rate = 0.001$

L2 损失：$lambda_loss = 0.005$

批处理参数：$batch_size = 200$

训练层数：$training_epochs = 300$

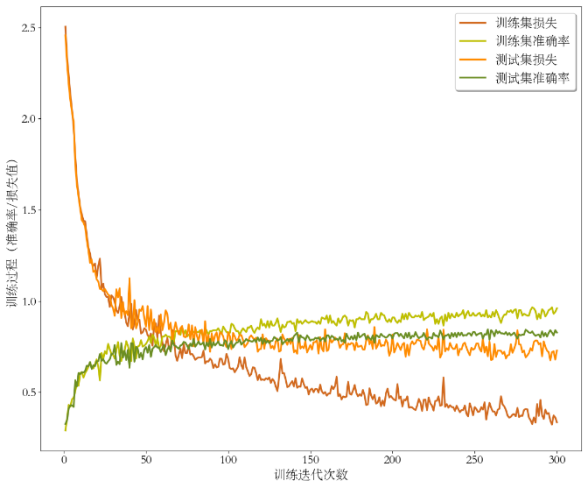
批处理参数：$batch_size = 200$

梯度裁剪：$clip_gradients = 15.0$

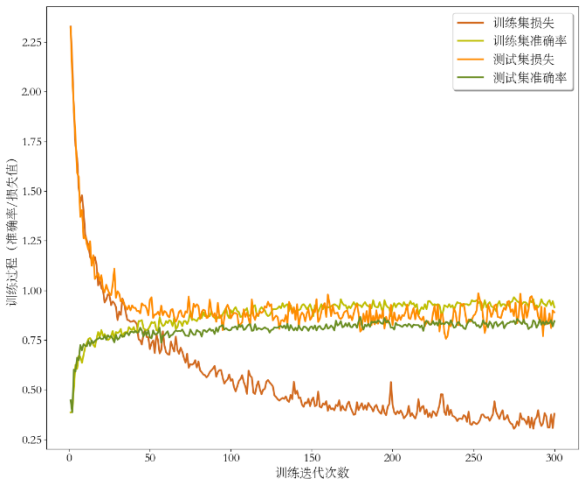
$Dropout$：$keep_prob_for_dropout = 0.85$

此处的$clip_gradients$参数的设置主要是为了解决梯度爆炸问题。

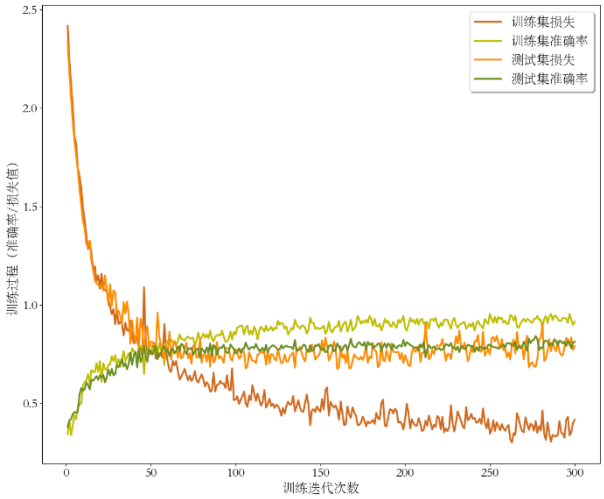
三种网络在训练验证过程中，训练集损失、训练集准确率、测试集损失、测试集准确率如下面三张图显示。



(a) Bidir-LSTM



(b)Res-LSTM



(c) DeepHAR

图 4.5：三个优化网络的训练效果

从图中可以注意到，三个模型对于基础的模型效果有显著的提升，尤其是训练集的损失，相对于基础的 LSTM 网络训练得到的结果有显著的降低。从模型收敛速度来说，Res-LSTM 模型收敛速度最快，最终的模型训练集损失也降低到了 0.25-0.3 之间。基础 LSTM 模型的测试集最终损失仍大于 1，而优化后的三个模型的测试集损失都大大降低，降到了 1 以下。相对来说，最终的 DeepHAR 模型的波动幅度较小在几个模型中是最小的，较为稳定。

从模型准确度角度来看，我们对比了四个网络的效果如表 4.1 所示：

表 4.2：网络效果对比

网络	达到最优效果的迭代次数	准确度	$F1-score$
基础 LSTM	>10000	81.153%	81.174
Bidir-LSTM	257	84.479%	84.321
Res-LSTM	276	84.082%	84.257
DeepHAR	174	86.696%	86.347

通过该表，我们可以比较得到不同网络的训练效果，根据表 4.2 的准确度对比，可以发现 DeepHAR 具有最高的准确度和$F1-score$，同时达到最优效果的迭代次数也是最少的。相比较而言，Bidir-LSTM 和 Res-LSTM 两个网络的效果就较为相似，但是其准确度远大于基础 LSTM 网络，证明通过增加堆叠层和增加残差网络促进网络通信对于增强网络训练效果来说是有效的。

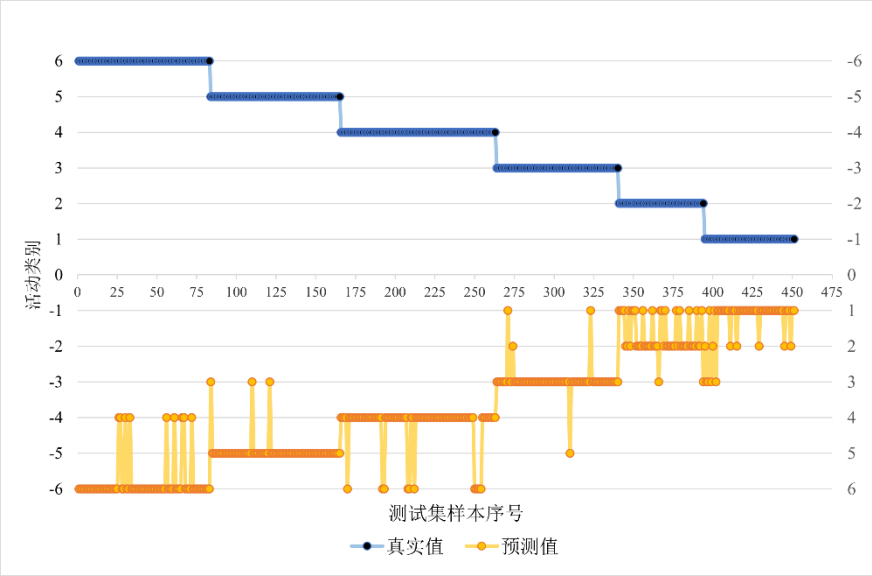


图 4.6：测试集样本预测值真实值对比

接着本文将对效果最优的网络的预测结果和真实结果进行分析，从图 4.6 易知，总体预测结果是符合真实值的，容易观察得到，在第一二类活动的判断上，数据的偏差率较高，也就是走路和走楼梯两个活动。从数据收集角度来说，由于每一层楼梯中间会有一部分空间是走路完成的，但是时间较短，很难排除中间走路动作对于数据的影响，因此这部分数据的偏差是可以理解的；从传感器角度来说，由于使用的数据主要是加速度计和陀螺仪，收集的数据没有高度计的内容，因此在上下楼梯时数据对于高度不敏感，易将走路和上下楼梯两个活动混淆。

通过预测值和正确值可以计算得到 DeepHAR 混淆矩阵式 4.8，行代表了真实的标签，列代表了预测的标签，根据矩阵可以得到图 4.7。

$$\begin{bmatrix} 47 & 6 & 4 & 0 & 0 & 0 \\ 18 & 35 & 1 & 0 & 0 & 0 \\ 2 & 1 & 73 & 0 & 1 & 0 \\ 0 & 0 & 0 & 83 & 0 & 15 \\ 0 & 0 & 3 & 0 & 79 & 0 \\ 0 & 0 & 0 & 9 & 0 & 74 \end{bmatrix} \tag{4.8}$$

矩阵中从左到右、从上到下的每一类依次为走路、上下楼梯、原地转圈、坐、站、躺。我们将其进行归一化处理后得到混淆矩阵如图 4.7 所示：

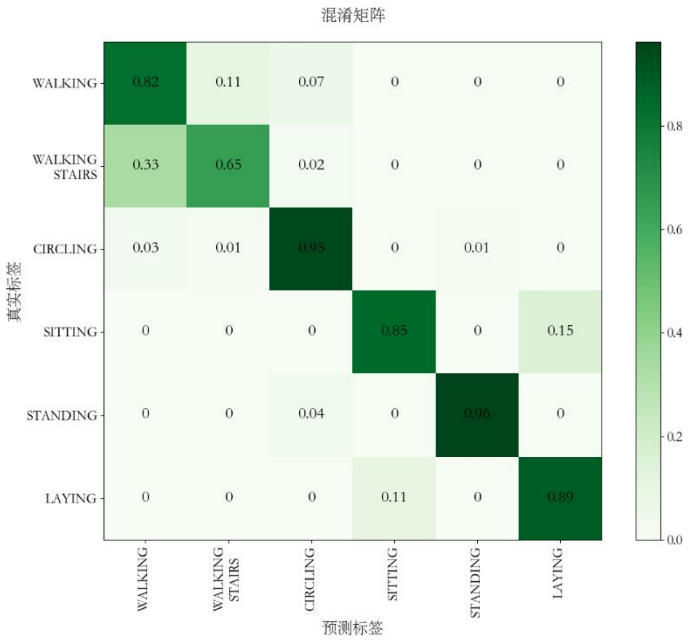


图 4.7：DeepHAR 网络结果的混淆矩阵

从图 4.7 中可以看出模型在转圈、坐、站、躺这四类状态取得较为优秀的分类效果，但是在走路和走楼梯这两类数据中，模型的分类效果较差，如果去除这两类，模型的分类成功率甚至可以达到 90%以上。与图 4.4 基础 LSTM 网络训练得到的结果相比，模型精度有非常大的提高，分类功能更加显著，第一类 walking 状态判断准确度提高了 20%，模型分类聚集更加明确，新的算法有效提升了算法的效果。

但是模型的准确度依然没有达到 90%以上，分析可能主要是数据本身走楼梯状态内夹杂走路状态导致的，但是以目前实验条件，这种情况较难排除，因此我们需要更加科学的公开数据集来检验模型的效果。

整体而言，我们的算法在智能手表数据集上还是表现得较为出色，在四种状态分类中能达到 90%以上的准确度，六种状态分类准确度能达到 86%，是科学且准确的算法。

4.3 算法在智能手机数据集上的效果

4.3.1 数据集分析

算法需要在更加科学的数据上得到检验，因此选择了公开的智能手机活动识别数据集。

在训练网络前，我们对输入的数据集基本情况进行分析如下：

输入传感器的测试数据共有 2947 条，训练数据共 7352 条，基本比例约为 7:3，智能手机的数据集由于是公开数据集，因此其区分是由该数据集的采集者进行分

配的。每条数据都已经做好裁剪，每条数据的长度为 128，即包含 128 个采样点，对应有 2947 条数据标签，数据标签共 6 种。测试数据传感器数据的平均值为 0.0991399，数据标签方差为 0.395671。

4.3.2 基础的 LSTM 算法

我们首先检验了基础 LSTM 在智能手机传感器数据集上的效果，同样也是只对基础 LSTM 添加了$L2$ loss 来防止神经网络对数据进行过度拟合。我们根据已有的论文并结合在智能手表上数据训练得到的参数来配置我们的模型[32]。由于该公开数据集相比自己搜集的智能手表数据集数据量更大，因此我们将会适当调低一些模型参数，在保证准确率的前提下节约内存空间。

学习率：$learning_rate = 0.0025$

L2 loss：$lambda_loss = 0.0015$

训练迭代次数：$training_iter = training_data_count * 300$

批处理参数：$batch_size = 1500$

由于基础 LSTM 的网络效果较差，因此我们需要训练大量次数才能达到相对满意的效果，设置的训练迭代次数约为 2205600 次，基本在 24000 次左右模型的准确率趋于稳步上升状态，如图 4.8 所示，相比于训练集较高的准确率和较低的损失，模型在测试集检验时的准确率相对较低，但也达到较不错的水平。

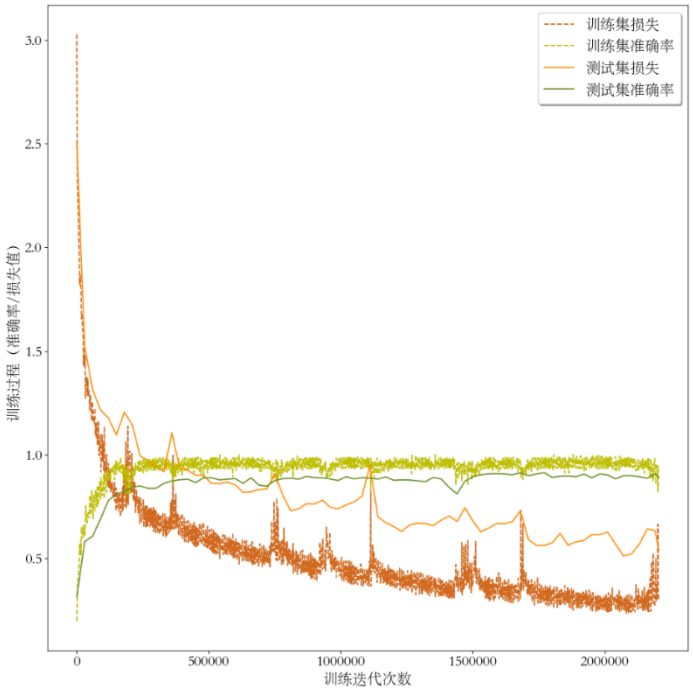


图 4.8：智能手机数据集在基础 LSTM 的训练效果

这次实验最终模型的精确度达到了 89.23938%，召回率为 89.00576%，最终获

得的$F1\ Score$为 88.95539，因此模型达到了较好的效果，但是相比于目前其他 90%以上评分的模型，还是存在一定差距。模型最后的批损失 Batch Loss 约为 0.64006，模型准确度 Accuracy 约为 0.89006，未达到 90%，与我们想要的正确率达到 90%以上的模型还存在差距，因此我们考虑优化模型。

本次训练初始的混淆矩阵值如下式所示：

$$\begin{bmatrix} 476 & 0 & 20 & 0 & 0 & 0 \\ 27 & 426 & 18 & 0 & 0 & 0 \\ 2 & 7 & 411 & 0 & 0 & 0 \\ 1 & 20 & 5 & 407 & 58 & 0 \\ 2 & 5 & 1 & 132 & 392 & 0 \\ 0 & 26 & 0 & 0 & 0 & 511 \end{bmatrix} \tag{4.9}$$

矩阵中从左到右、从上到下的每一类依次为走路、上楼梯、下楼梯、坐、站、躺。我们将其归一化处理后得到矩阵：

$$\begin{bmatrix} 0.95967 & 0 & 0.04032 & 0 & 0 & 0 \\ 0.05732 & 0.90445 & 0.03821 & 0 & 0 & 0 \\ 0.00476 & 0.01666 & 0.97857 & 0 & 0 & 0 \\ 0.00213 & 0.04273 & 0.01068 & 0.86965 & 0.12393 & 0 \\ 0.00373 & 0.00934 & 0.00186 & 0.24672 & 0.73271 & 0 \\ 0 & 0.04841 & 0 & 0 & 0 & 0.95158 \end{bmatrix} \tag{4.10}$$

图 4.11(a)展示了混淆矩阵的可视化效果。从其中可以看出模型整体的分类效果较为显著，但是在上下楼梯、走路方面，出现了一定的误差，而站和坐也存在较大的偏差，这可能是因为手机传感器在这两个动作时，加速度差异和方向差异均不是很大造成的，但是预期站和坐可能会存在更大差异，因此模型的性能还是超出了预期。

目前的网络是利用了加速度计和陀螺仪数据训练得到的，当我们只是用加速度计的结果时，会发现最终的准确度只有 87%左右，证明单独使用加速度计的数据是不够准确的，在后面的实验中，我们都会利用加速度计和陀螺仪两个传感器的结果进行训练。接下来，我们将利用优化网络，取得更好的效果。

4.3.3 优化算法

由于基础的 LSTM 性能较差，我们继续尝试优化算法，根据 3.2 中的算法内容，向基础的 LSTM 添加了堆叠层变为了 Bidir-LSTM 网络、添加了残差网络变为 Res-LSTM 网络，并在最后将残差连接添加到了双向 LSTM 网络上构建了最后的 DeepHAR 网络来查看效果。对于优化后的三个模型，我们模型参数设置如下[32]：

学习率：$learning_rate = 0.001$

L2 损失：$lambda_loss = 0.005$

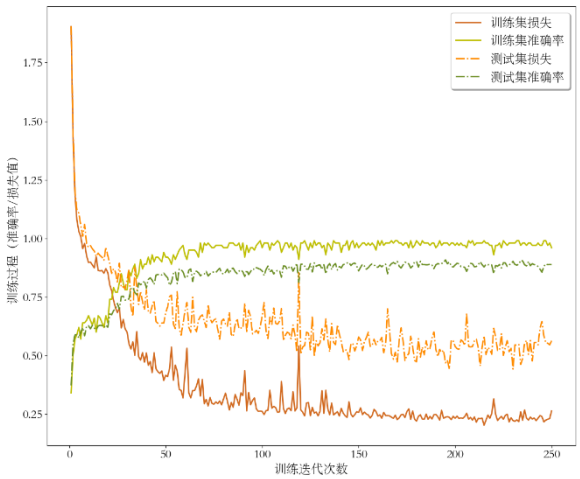
批处理参数：$batch_size = 100$

训练层数：$training_epochs = 250$

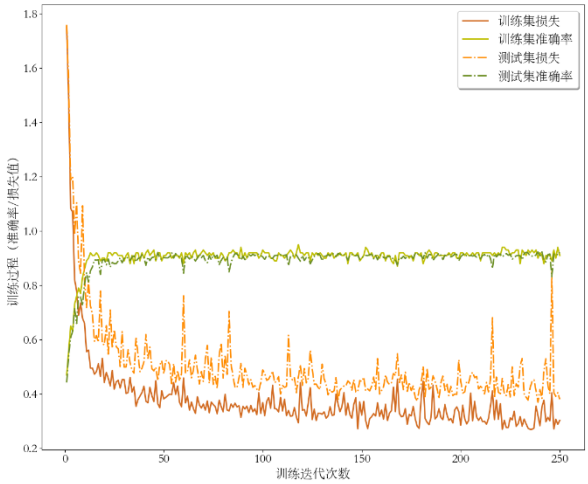
梯度裁剪：$clip_gradients = 15.0$

$Dropout$：$keep_prob_for_dropout = 0.85$

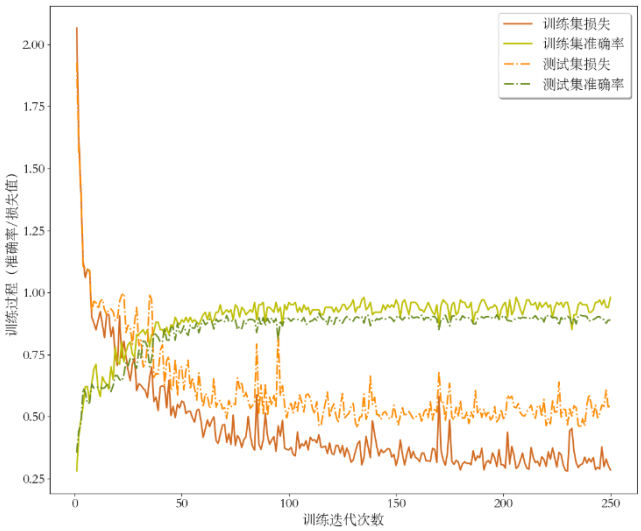
三种网络在训练验证过程中，训练集损失、训练集准确率、测试集损失、测试集准确率如下面三张图显示。



(a) Bidir-LSTM



(b)Res-LSTM



(c) DeepHAR

图 4.9：智能手机数据集在三种网络的训练效果

比较图 4.9 中三个模型的结果，可以发现准确度的差异很难在图上反映出来，因此我们将在后文进行详细讨论。但是图中值得注意的是，最终的 DeepHAR 模型实际上相比于其他两个模型，训练集和测试集的准确度差异和损失差异都更小，其训练结果在训练集和测试集上都取得了较好的效果。同时，其震荡幅度也是最小的，

在迭代次数 250 次以上时，网络的准确度就基本保持稳定波动上升，损失震荡幅度较小，最终的在测试集上的损失也较小，基本最终处于 0.5 左右，其他两个网络的测试集上的损失曲线震荡研究，临近两次迭代的极差甚至达到了 0.5 以上。接着进行分析 Bidir-LSTM 网络的结果，可以从图中看出，网络在训练集和测试集损失差距较大，基本差距 0.4-0.5 左右，同时损失波动也很大。准确率的上升并非稳步上升，而是在前几个 epoch 训练的精度较低，在后期准确度才缓缓升高，相比于其他两个网络，模型的精度和模型训练效率都有待提升。对于 Res-LSTM 网络，其精度波动相对较小，并且模型的收敛速度是三个模型中最快的，但是其损失曲线的震荡幅度较大。

当训练结束时，训练和测试结果都在一个固定值附近振荡。每组的结果是趋同的，相比较上一章节基础 LSTM 的震荡幅度，可以发现其明显高于优化后的三个网络。三个模型收敛速度可以从慢到快排列为基线 LSTM、Bidir-LSTM、Res-LSTM 和 DeepHAR。但是，DeepHAR 和 Res-LSTM 之间的差异非常小，而远优于其他模型。这充分证明了，残差连接在收敛性方面表现突出。

从模型准确度角度来看，我们对比了四个网络的效果如下表所示：

表 4.3：网络效果对比

网络	达到最优效果的迭代次数	准确度	$F1-score$
基础 LSTM	>>300	89.006%	88.955
Bidir-LSTM	194	90.786%	89.836
Res-LSTM	113	91.598%	91.584
DeepHAR	238	93.347%	93.178

对比网络效果可以发现，修改过后的网络架构在智能手机数据库上的效果都优于基础的 LSTM 网络。其中，Bidir-LSTM 和 Res-LSTM 的性能较为相近且都较为擅长信息传输。 Bidir-LSTM 可以在前向和后向传播中获取信息，而 Res-LSTM 可以使用残差网络直接传输信息。最终的网络 DeepHAR 拥有残差连接和双向传输的网络，取得了最高的$F1$分数，为 93.1780%。我们在训练的时候是随机选择了一个 batch，一个完整的计算几乎可以遍历整个数据集。

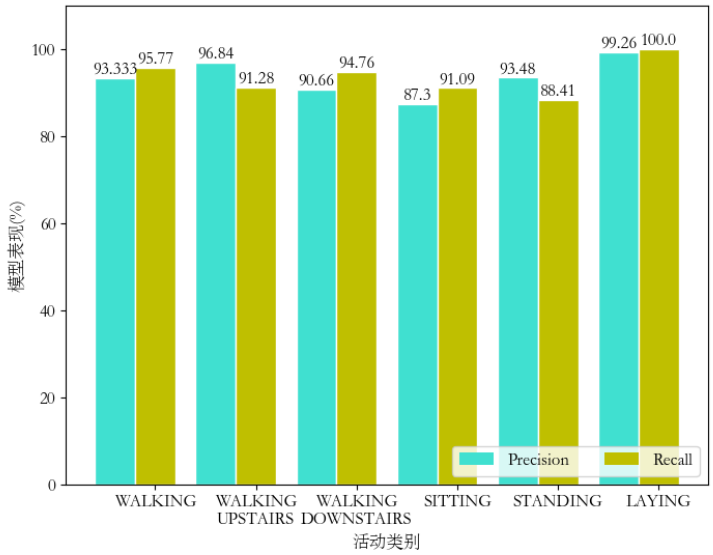
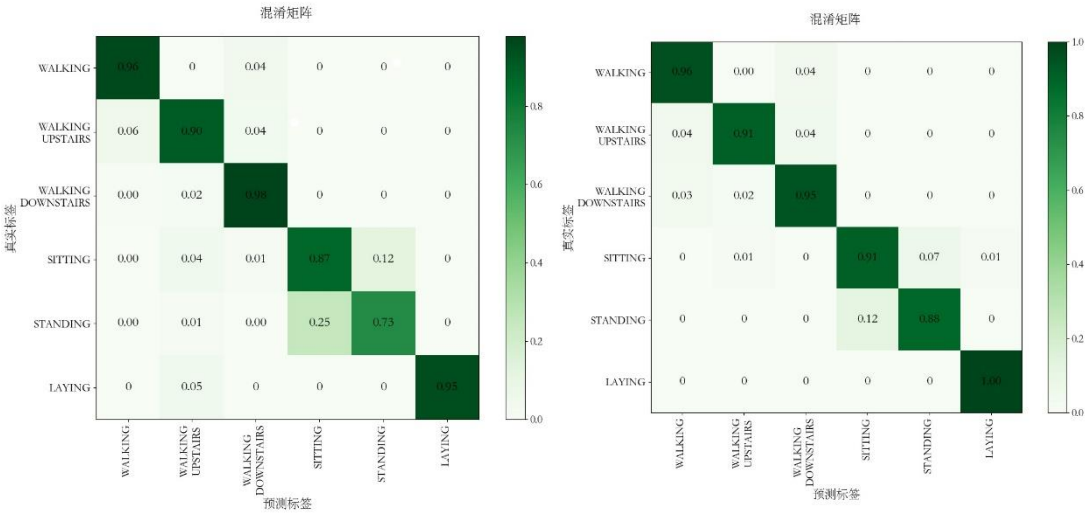


图 4.10：不同活动类别的识别表现

最后，我们为 DeepHAR 的每个活动类别计算了其 Precision 和 Recall 值，正如图 4.10 所展示的。同时我们根据每个活动类别的识别情况，计算得到了混淆矩阵，并绘制了图 4.11(b)更清晰地显示各个标签的预测情况。从中，我们可以看到“SITTING”和“STANDING”两个较为相似的标签之间之间的预测存在一定的混淆，根据智能手机数据集最初收集数据的方式，手机都放置在腰部，传感器的参数对这两个状态来说应该是基本相同的，但是仍然有将近 90%的数据都预测成功，区分出了这两个状态。在“WALKING”、“WALKING_UPSTAIRS”和“WALKING_DOWNSTAIRS”三个状态之间也存在一定混淆，这三个状态在动作和行为上非常相似，因此传感器数据也会较为相似，因此这三类状态之间的错误辨别和在混淆矩阵上的表现是可以理解的。图展示了最终 DeepHAR 的混淆矩阵，与初始 LSTM 的混淆矩阵相比较，可以明显看出新的模型更容易区分站和坐的活动状态，同时走路和走楼梯的混淆程度也有了大大降低。图中对角线相比于初始模型颜色加深，即模型的正确率提高了，分散在错误类别的数据大大减少。因此事实上模型在区分活动状态方面，有了显著的性能提升。



(a) 基础 LSTM 的混淆矩阵 (b) DeepHAR 的混淆矩阵

图 4.11：两种网络在智能手机数据集上的效果对比

4.4 算法效果比较

4.4.1 两个数据集的结果比较

在这一部分我们将主要比较最优的 DeepHAR 在两个不同数据集的效果。可以从 4.2 和 4.1 得到，模型在自己采集的智能手表数据集上取得的模型准确度约为 86.347%，而在公开的 UCI HAR 数据集中则取得了 93%的准确度，虽然准确度不同，但是对于基础的 LSTM 模型，优化的网络准确度是有显著提高的，因此我们优化网络提升了网络的性能是毋庸置疑的，其结构能够适应手机和手表的数据，根据提供的数据适应不同的环境。可以留意到，在智能手表的原地转圈、坐、站、躺四种活动状态中，模型的准确度达到了 90%以上，因此模型表现出了较好的表现。在智能手表中数据集中难以辨别走路和走楼梯的原因是目前实验环境很难排除走楼梯过程中在连接处走路的影响，同时没有添加高度计数据，难以更准确判断高度提升这一状态。因此，事实上在智能手表数据集中模型的表现相对智能手机表现较差的原因是与数据粗糙性有关，而与模型本身没有关系。模型的优越性和鲁棒性可以从同一数据集准确度的提升中反映出来。

4.4.2 其他结果比较

为了进一步验证模型的效果，我们将模型效果与其他学者的研究成果进行比较。由于智能手表数据集由自己设计，无法找到匹配的结果，因此我们只对比 UCI HAR 数据集上的结果。多数学者在研究人类活动识别时，都会选择这一数据集进行实验。

从表 4.4 中可以看出，本文提出的 DeepHAR 模型在性能方面略优于其他学者

提到的模型，相对于基于机器学习的朴素贝叶斯、SVM 等其他模型，利用深度学习的 LSTM 模型在性能上远远超过了这些模型，足以证明文中算法具备良好的性能。同时即使是在性能较差的机器上，模型的运行速度也较快，具备良好的未来实用价值。

表 4.4：不同网络在 UCI HAR 数据集上的效果对比

模型	准确率(%)
朴素贝叶斯_FW[36]	90.10
KNN_FW[36]	87.80
HCF+ANN[37]	91.08
MC-SVM[38]	89.30
Bidir-LSTM	90.79
Res-LSTM	91.60
DeepHAR	93.34

从结果中看，基于时间序列的深度学习 LSTM 模型在人体活动识别中具有非常优秀的表现。一部分是时间序列在人体活动识别的重要性导致的。因为人类的活动必然是基于时间发生的，因此其动作的判断也必然与某一时刻前后的动作序列有关。只通过单一数据点进行的判断难以完整地判断人的整个活动。因此在构建模型时，需要选取合适的时间序列进行动作判断。本次实验中选取的时间序列是 128 个时间步长，充分考虑了已有文献和实际的实验需要，调整时间序列的长度也必然会影响模型判断人体活动的精度。总体而言，我们的模型成功将活动识别的准确率提高了 3%-5%，是切实有效的模型，即使在每个数据集我们选取的六个活动类别中都涉及一些相似容易存在混淆的行为，但是，仍然可以在矩阵中看到这三个类之间的一些聚类，模型仍然可以有效区别出不同的活动类别。

5 结论与展望

本文分析了 HAR 研究的意义，并概述了该领域的新兴方法，总结提出了 DeepHAR 网络的新框架来进行活动识别的研究。事实上，LSTM 神经网络作为一种可以有效处理时间序列的网络，已经被用于多个领域如自然语言处理、语音识别和天气预报中，当然该技术也适用于活动识别的任务。这种深度网络可以增强学习能力，从而在早期训练中更快地学习。我们提出的网络还通过增加残差连接（在深度维度上）和双向单元（在时间维度上）多重保证了信息传输的有效性，提高了模型的有效性。与之前的工作相比，我们的实验所提出的网络能够将公共数据集 UCI HAR 的智能手机传感器数据集的识别准确率提高 5.54%，有效提升了识别的精度。

在实验中，我们也为模型添加了多个超参数，例如学习率和 $L2$ 权重衰减乘数。所有的超参数调试都必须在网络架构完成后进行。模型中各个超参数的取值、以及区分每一次活动的时间序列长度也需要进行实时的调整，因此这增加了整体复杂性并且增加了工作的复杂度。整体而言，我们找到了最适合两个数据集的超参数和窗口大小。

在研究了新的有效算法之外，我们也开发了一个适用于智能手表内部传感器自动收集的 APP，通过该 APP 可以以不同频率收集一分钟内的人类活动导致的传感器数据变化，并且程序将会自动为数据加上标签，存储至电脑，便于进行下一步工作。该程序弥补了目前智能手表传感器数据收集程序的不足，有利于未来对人类活动数据在智能手表上的研究，也为我们的研究奠定了数据基础。

对于未来的工作，由于超参数调整的复杂性，我们十分希望探索一种更有效的方法来调整参数。虽然我们在实验中用到了网格搜索的方法，但是其搜索范围需要手动更改，且值始终是固定的。未来我们考虑寻找一种自适应方式来自动调整搜索过程，并能够实现使神经网络的架构进行自动调整，例如自动重塑、添加和删除各个层。除此之外，对于 LSTM 模型本身，我们可以进行更加深入的探索，考虑在 LSTM 单元中的一个或某些点混合一些基于时间的卷积的效果来改善结果。当然，作为一种网络，LSTM 网络也有其缺点，其允许在任意的时间延迟中存储信息，而错误的内容可能会在很长的时间内被带回。这种潜在的强度在某些情况下会导致一个弱点：在时间序列的表示过程中，单元状态记录器趋向于线性增长。如果我们呈现连续的输入流，单元状态可能会无界地增长，从而导致输出压缩函数的饱和，使整个存储单元将被分解为一个普通的 bptt 单元[40]。因此，我们将致力于继续优化 LSTM 网络，提高其在活动识别方面的有效性。

从研究结果来看，目前我们构建的深度学习网络的性能依然依赖于标记样本的数量和质量，但是获取足够的活动标签既昂贵又耗时，因此，未来对于如何利用少量的样本，得到适用性较广的结论，将是项目研究的重要内容之一。

最后，我们也可以考虑将 DeepHAR 网络应用在其他的领域。作为一个表现较为优秀的模型，其应该具有出色的泛化能力，因此可以考虑将我们的算法应用在其他时间序列问题中，比如一些库存预测和轨迹预测的问题。结合新的问题可以从其他方面找到模型的不足并进行下一步修改。

致 谢

从季夏到孟夏，四载光阴匆匆而过，在重庆大学的四年不仅仅是青春中最美好的四年，更是充斥着知识、感动、努力和泪水的四年。

在进校门时对即将迈入的新的领域的未知和懵懂，到现在的坚定和坦然，改变的不仅是对于更加高阶知识的认识，更是对于世界、对于人生的感悟。四年来不能说对于某个领域的知识有了真正深入的了解，但是终于领悟了学习方法和学习思路。现在的我还站在一个宽阔而看不见边界领域的起点，希望着未来能够到达有一定深度的地方，我将这一切的可能交给未来。

感恩二十年来一直照料我、无条件爱我的妈妈和外婆，感谢她们永远在我的背后支持我的所有决定，做我最最最坚实的后盾，让我在学习的同时没有后顾之忧，在痛苦的时候有人可以依赖。

感谢我的老师古富强老师，他为我指明了毕设的方向，并从开题报告到论文终稿的一次次修改中不厌其烦指出问题。他让我领会科研的步骤，使我对崭新的领域慢慢变得熟悉，并让经常拖延到最后一刻的我提高了效率。我将铭记在心古老师丰富的学术知识和耐心教学的态度。

感谢四年来大学中的每一位老师，他们领我进知识的大门，让我坚定了未来的方向，踏实地走在求学的道路上。感谢在我求学道路上每一位帮助过我的人。如果没有老师带给我的知识、没有朋友们带给我的快乐，这篇论文的完成和我的大学四年相信都不会如此顺利和幸福。

要感谢的人太多，难以一一详述。就最后感谢四年里一直没有放弃的自己，不知道结局会是如何，但是希望这会是没有遗憾的四年。希望自己未来不要忘记自己现在的目标与梦想，永远保持良好的心态，“吟啸且徐行”，不留遗憾走下去。

禹门鱼变辞凡水，乔水莺迁出故林。寄语至公堂里客，莫教冰鉴负初心。

搁笔的这一刻，是结束，也是开始，这是一趟完整的旅程，每个人下车的时候，也一定会抵达想去的终点。

参 考 文 献

[1] Wang J , Chen Y , Hao S , et al. Deep Learning for Sensor-based Activity Recognition: A Survey[J]. Pattern Recognition Letters, 2017.

[2] Gu F , Khoshelham K , Valaee S , et al. Locomotion Activity Recognition Using Stacked Denoising Autoencoders[J]. IEEE Internet of Things Journal, 2018:1-1.

[3] Dang L M , Min K , Wang H , et al. Sensor-based and vision-based human activity recognition: A comprehensive survey[J]. Pattern Recognition, 2020, 108(107561).

[4] Vepakomma P , D De, Das S K , et al. A-Wristocracy: Deep Learning on Wrist-worn Sensing for Recognition of User Complex Activities[C]. IEEE Body Sensor Networks Conference. IEEE, 2015.

[5] Qin J , Liu L , Zhang Z , et al. Compressive Sequential Learning for Action Similarity Labeling[J]. IEEE Transactions on Image Processing, 2015, 25(2):756-769.

[6] Hammerla N Y , Halloran S , Ploetz T . Deep, Convolutional, and Recurrent Models for Human Activity Recognition using Wearables[J]. Journal of Scientific Computing, 2016, 61(2):454-476.

[7] Kim Y , Toomajian B . Hand Gesture Recognition Using Micro-Doppler Signatures With Convolutional Neural Network[J]. IEEE Access, 2016, 4:7125-7130.

[8] Poppe R . A survey on vision-based human action recognition[J]. Image & Vision Computing, 2010, 28(6):976-990.

[9] Wang S , Zhou G . A review on radio based activity recognition[J]. Digital Communications & Networks, 2015, 1(1):20-29.

[10] Cook D , Feuz K D , Krishnan N C . Transfer Learning for Activity Recognition: A Survey[J]. Knowledge & Information Systems, 2013, 36(3):537-556.

[11] Zhu, Fan, Shao, et al. From handcrafted to learned representations for human action recognition: A survey.[J]. Image & Visionuting, 2016.

[12] Chen L , Hoey J , Nugent C D , et al. Sensor-Based Activity Recognition[J]. IEEE Transactions on Systems Man & Cybernetics Part C, 2012, 42(6):790-808.

[13] Shoaib M , Bosch S , Incel O , et al. A Survey of Online Activity Recognition Using Mobile Phones[J]. Sensors, 2015, 15(1):2059-2085.

[14] Gu F , Kealy A , Khoshelham K , et al. User-Independent Motion State Recognition Using Smartphone Sensors[J]. Sensors, 2015, 15(12):30636-30652.

[15] Gu F , Chung M H , Chignell M , et al. A Survey on Deep Learning for Human Activity Recognition[J]. ACM Computing Surveys, 2021.

[16] Sigg S , Blanke U , Troster G . The telepathic phone: Frictionless activity recognition from

WiFi-RSSI[C]. IEEE International Conference on Pervasive Computing & Communications. IEEE Computer Society, 2014:148-155.

[17] Li X , Zhang Y , Marsic I , et al. Deep Learning for RFID-Based Activity Recognition[C]. The 14th ACM Conference on Embedded Networked Sensor Systems (SenSys 2016). ACM, 2016.

[18] Ball S , Saba E A , Peker M . Human activity recognition from smart watch sensor data using a hybrid of principal component analysis and random forest algorithm[J]. Measurement and Control - London- Institute of Measurement and Control-, 2019, 52(3):002029401881369.

[19] Lara, Oscar D , Labrador, et al. A Survey on Human Activity Recognition using Wearable Sensors[J]. IEEE Communications Surveys & Tutorials, 2013, 15(3):1192-1209.

[20] Bao L , Intille S S . Activity Recognition from User-Annotated Acceleration Data[C]. Pervasive Computing, Second International Conference, PERVASIVE 2004, Vienna, Austria, April 21-23, 2004, Proceedings. Springer, Berlin, Heidelberg, 2004.

[21] Hu L , Chen Y , Wang S , et al. Less Annotation on Personalized Activity Recognition Using Context Data[C]. 2016 Intl IEEE Conferences on Ubiquitous Intelligence & Computing, Advanced and Trusted Computing, Scalable Computing and Communications, Cloud and Big Data Computing, Internet of People, and Smart World Congress (UIC/ATC/ScalCom/CBDCom/IoP/SmartWorld). IEEE, 2017.

[22] Lecun Y , Bengio Y , Hinton G . Deep learning[J]. Nature, 2015, 521(7553):436.

[23] Lane N D , Georgiev P , Qendro L . DeepEar: Robust Smartphone Audio Sensing in Unconstrained Acoustic Environments using Deep Learning[C]. UbiComp '15 ACM International Joint Conference on Pervasive and Ubiquitous Computing - September 07 - 11, 2015. ACM, 2015.

[24] Thomas Plötz, Nils Y. Hammerla, Patrick Olivier. Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence Feature Learning for Activity Recognition in Ubiquitous Computing[C]. 2012.

[25] Bengio Y . Deep Learning of Representations: Looking Forward[J]. Springer Berlin Heidelberg, 2013.

[26] Zhou B , J Yang, Li Q . Smartphone-Based Activity Recognition for Indoor Localization Using a Convolutional Neural Network[J]. Sensors, 2019, 19(3).

[27] Inoue M , Inoue S , Nishida T . Deep recurrent neural network for mobile human activity recognition with high throughput[J]. Artificial Life and Robotics, 2018, 23(2):173-185.

[28] Bzarg. How a Kalman filter works, in pictures [EB/OL]. [2021-12-21]. http://www.bzarg.com/p/how-a-kalman-filter-works-in-pictures/#mjx-eqn-kalunsimplified.

[29] Forouzan, B.A. (2000). TCP/IP: Protocol Suite (1st ed.)[M]. New Delhi, India: Tata McGraw-Hill Publishing Company Limited, 2000.

[30] Bengio, Y, Simard, et al. Learning long-term dependencies with gradient descent is difficult[J]. Neural Networks IEEE Transactions on, 1994.

[31] Sensor Event [EB/OL]. [2022-05-15]. https://developer.android.com/reference/android/hardware/SensorEvent#values.

[32] Zhao Y， Yang R， Chevalier G，et al. Deep Residual Bidir-LSTM for Human Activity Recognition Using Wearable Sensors[J]. 2017.

[33] Hochreiter, Sepp, Schmidhuber, et al. Long short-term memory.[J]. Neural Computation, 1997.

[34] Anguita D， Ghio A， Oneto L，et al. A public domain dataset for human activity recognition using smartphones[C]. 2013.

[35] Pascanu R， Mikolov T， Bengio Y . Understanding the exploding gradient problem[J]. Arxiv Preprint Arxiv, 2012.

[36] Wang A， Chen G， Jing Y，et al. A Comparative Study on Human Activity Recognition Using Inertial Sensors in a Smartphone[J]. IEEE Sensors Journal, 2016, 16(11):4566-4578.

[37] Ronao, Charissa, Ann, et al. Human activity recognition with smartphone sensors using deep learning neural networks[J]. Expert Systems with Application, 2016.

[38] Anguita D， Ghio A， Oneto L，et al. Energy efficient smartphone-based activity recognition using fixed-point arithmetic[J]. Journal of Universal Computerence, 2013, 19(9):1295-1314.

[39] 卡尔曼滤波算法-Kalman Filter Algorithm [EB/OL]. [2022-05-17]. https://blog.csdn.net/sunlin972913894/article/details/104833610.

[40] Huang Z， Wei X， Kai Y . Bidirectional LSTM-CRF Models for Sequence Tagging[J]. Computer Science, 2015.